

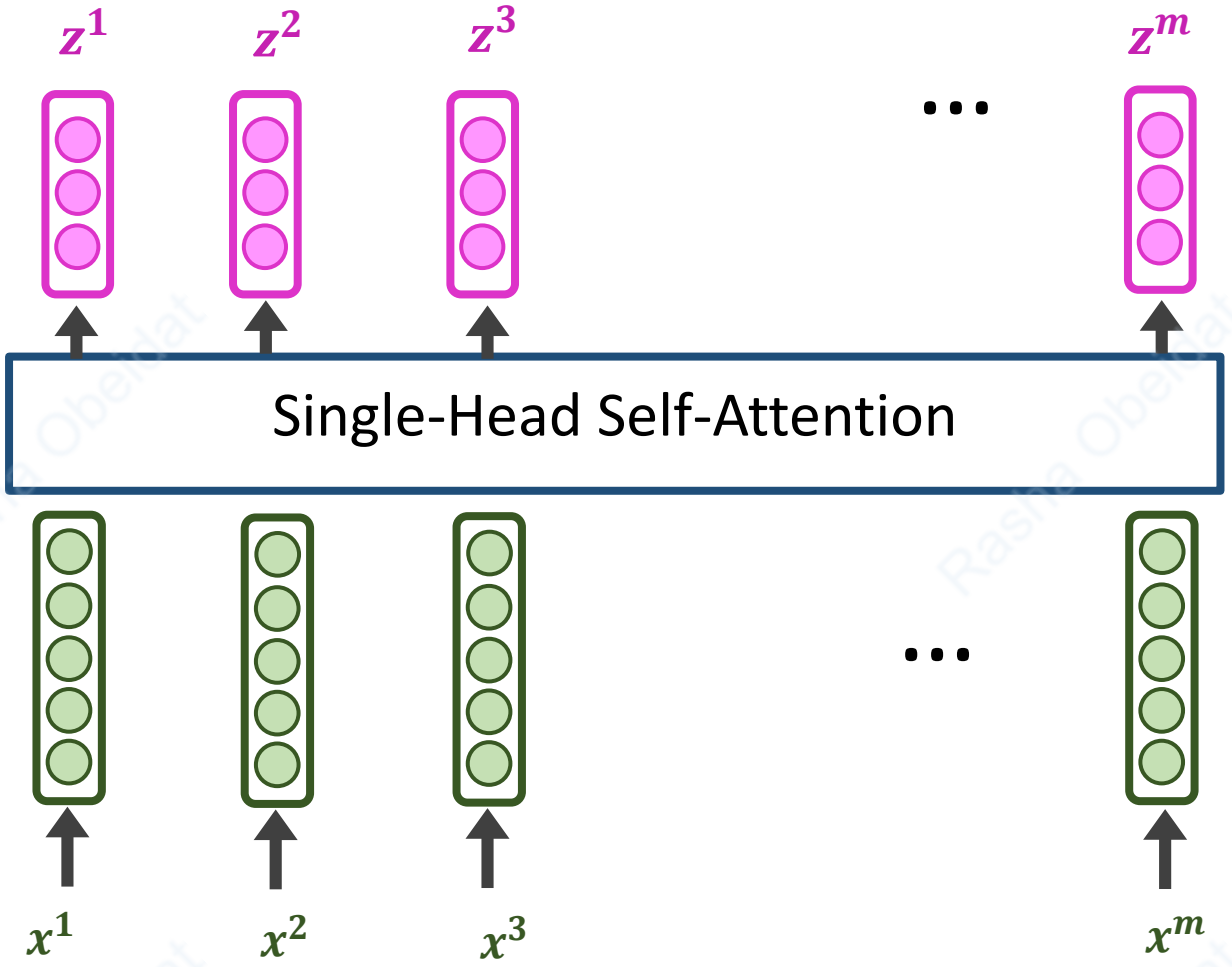
Transformers

Rasha Obeidat, PhD

AI445: NLP

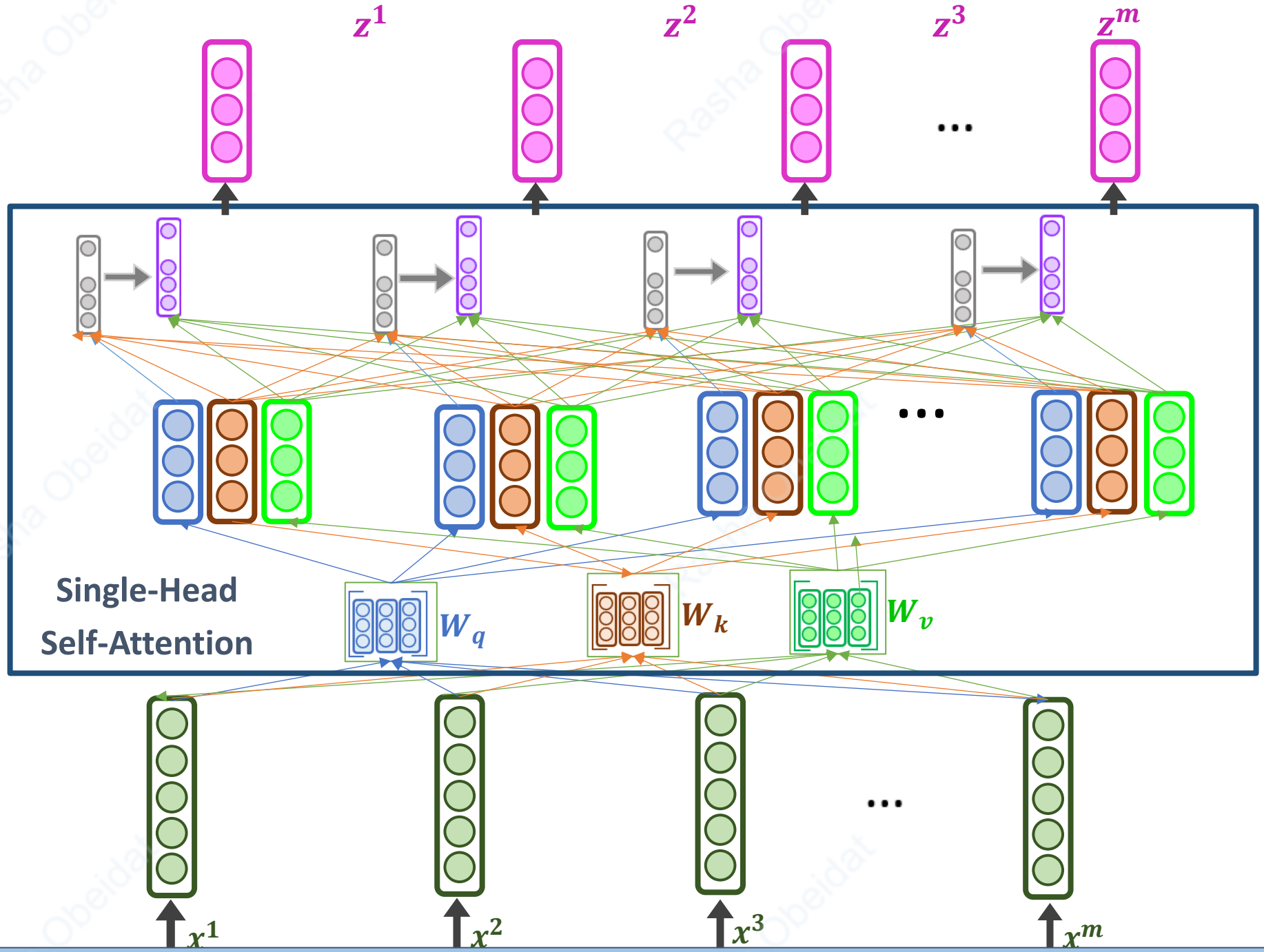
Rasha Obeidat

Self-Attention



- we learned about, last lecture, is a ***Single Head self-attention unit*** !!!!

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$



Single-Head Self-Attention

Moving from single head to multi head Self-Attention

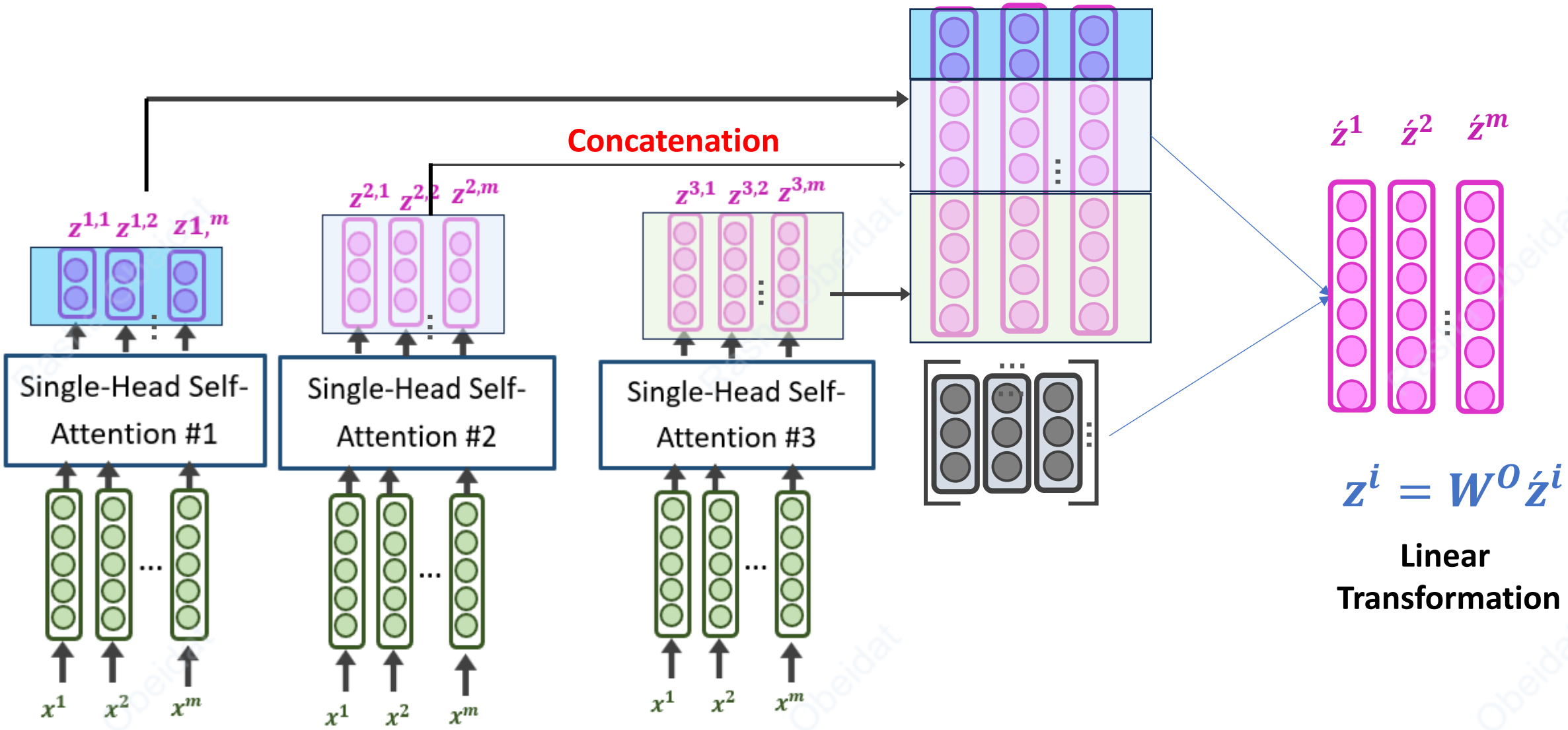
- **Single-Head Attention**:
  - Uses one set of Query, Key, and Value matrices to compute attention.
  - Focuses on a single aspect or pattern in the data at a time.

Rasha Obeidat

Moving from single head to multi head Self-Attention

- **Single-Head Attention**:
  - Uses one set of Query, Key, and Value matrices to compute attention.
  - Focuses on a single aspect or pattern in the data at a time.
- **Multi-Head Attention**:
  - It is found that it beneficial to linearly project the **queries**, **keys** and **values** $\boldsymbol{h}$ times with different, learned linear projections to $\boldsymbol{d^k}$, $\boldsymbol{d^k}$ and $\boldsymbol{d^v}$ dimensions, respectively.
  - On each of these projected versions of *passes* the input into multiple heads and computes attention independently for each head.
  - On each of these projected versions of queries, keys and values, multiple single-head attention are performed in parallel.
  - Each head has its own **Query**, **Key**, and **Value** matrices (probability with different dimensions (various $\boldsymbol{d^k}$ and $\boldsymbol{d^k}$ values between heads) , allowing the model to focus on different aspects of the input.
  - Outputs from all heads are ***concatenated*** and passed through a linear transformation.

Moving from single head to multi head Self-Attention



Moving from single head to multi head Self-Attention

- **Benefits of Multi-Head Attention**:
  - Captures diverse patterns and relationships in the data.
  - Improves model capacity and performance by leveraging multiple perspectives.

Rasha Obeidat

Moving from single head to multi head Self-Attention

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

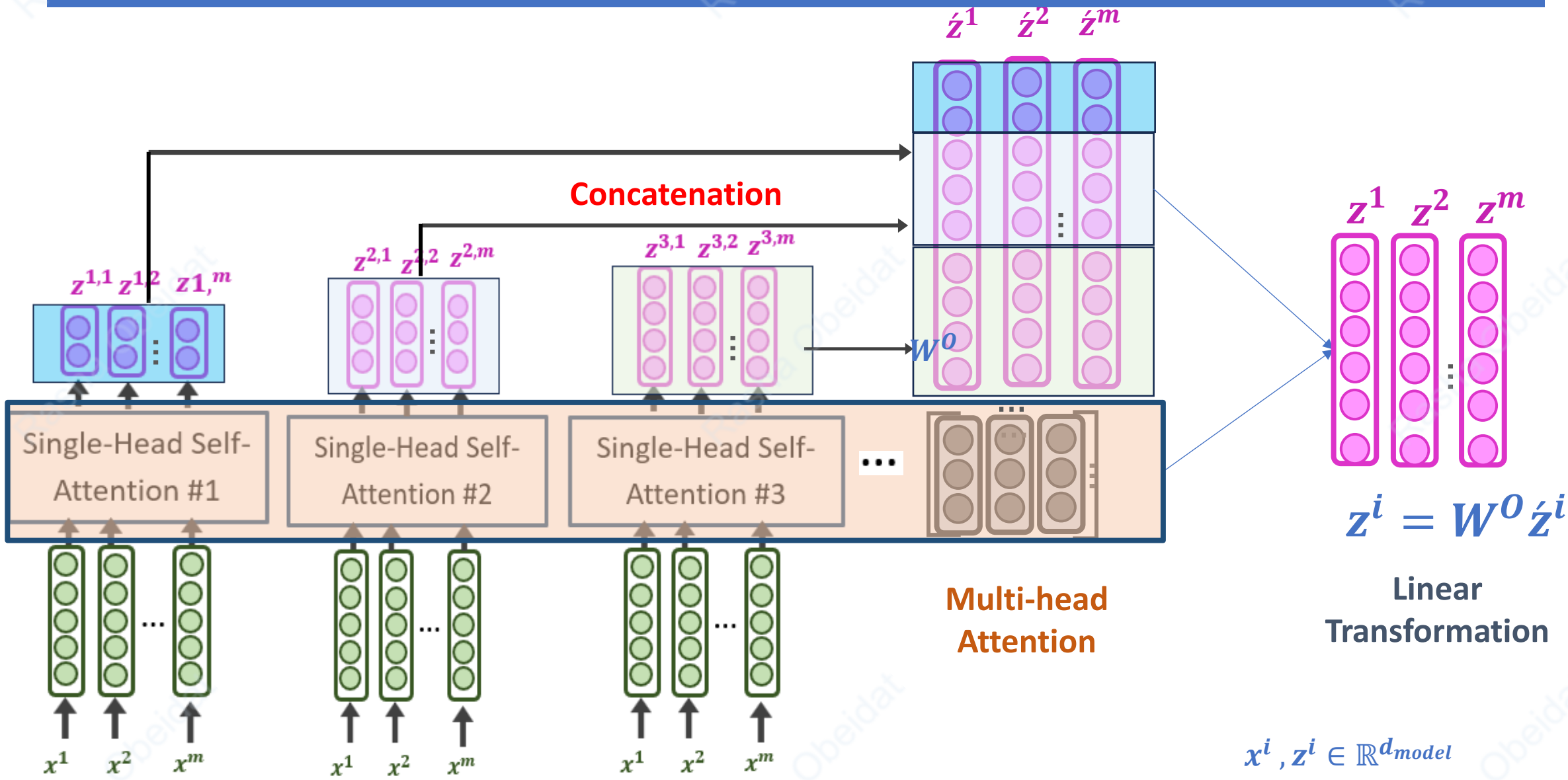


$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_\text{h})W^O$$
$$\text{where head}_\text{i} = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

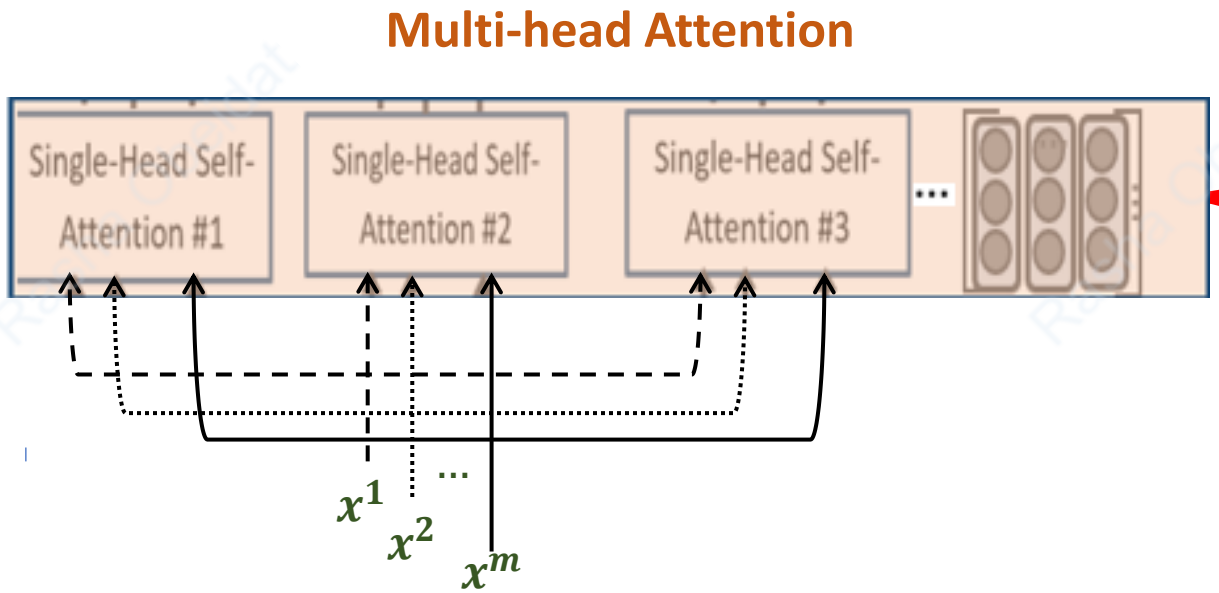
Single-head Attention

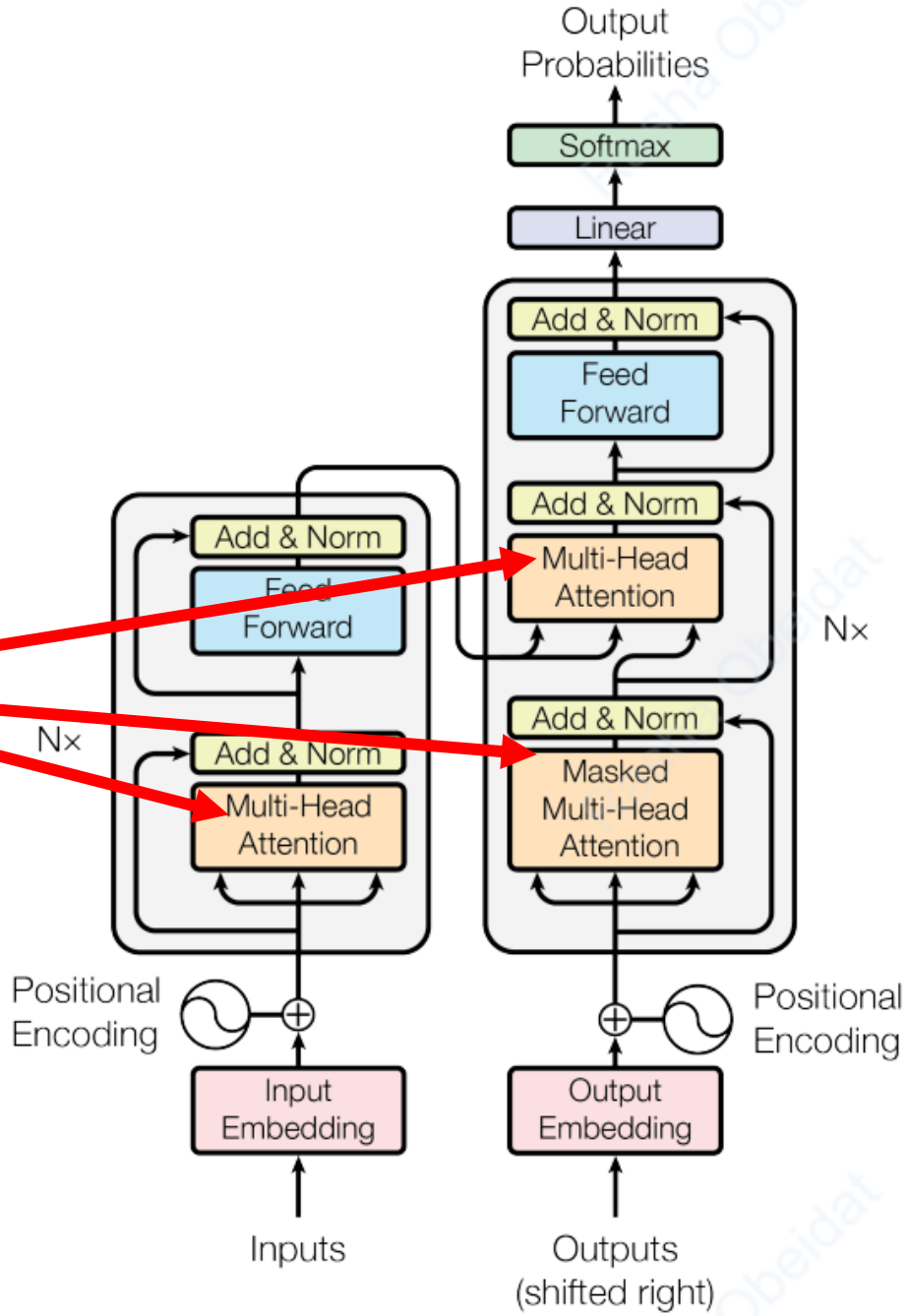
Multi-head Attention

Moving from single head to multi head Self-Attention



Moving from Multi head Self-Attention To *Transformer*

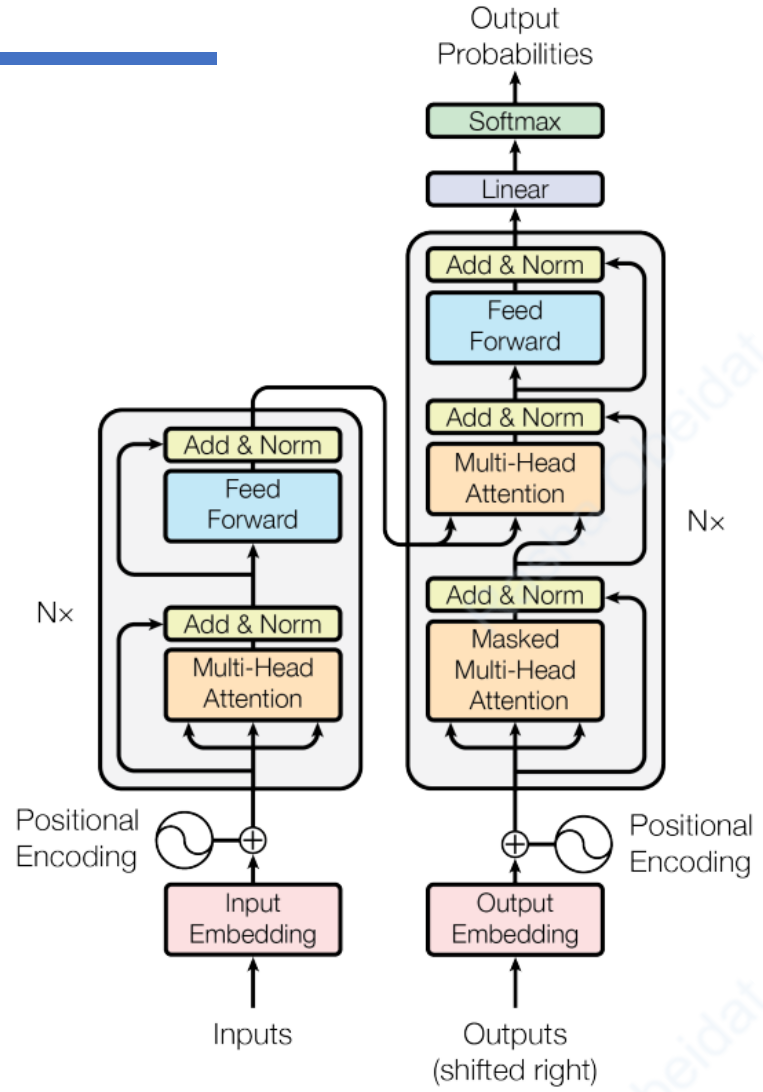




Multi-head Attention is one of many building blocks of ***Transformer***

Transformer

- **What is a Transformer?**
  - A transduction model that relies entirely on **self-attention** to compute input and output representations.
  - Eliminates the need for sequence-aligned RNNs or convolution layers, enabling parallel computation.
  - The **Transformer** was originally proposed for **MT** in the paper *"Attention is All You Need"* (Vaswani et al 2017).
  - It has since become the foundation for the recent sequence generation models.



Transformer

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

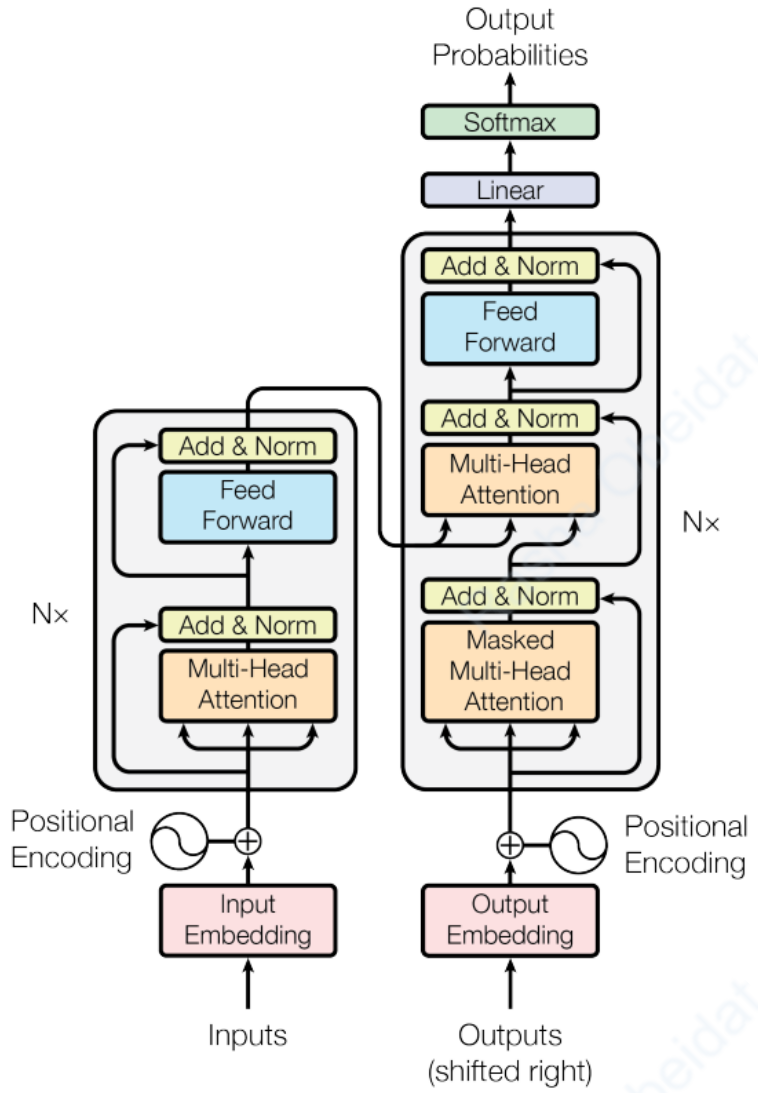
Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

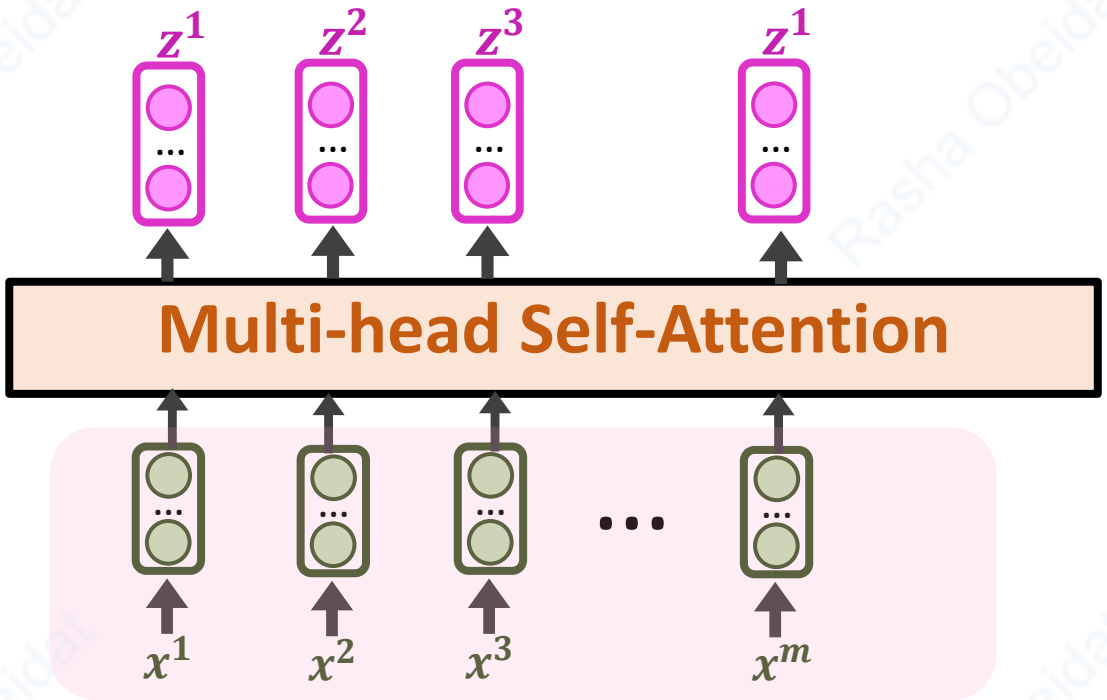
Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com



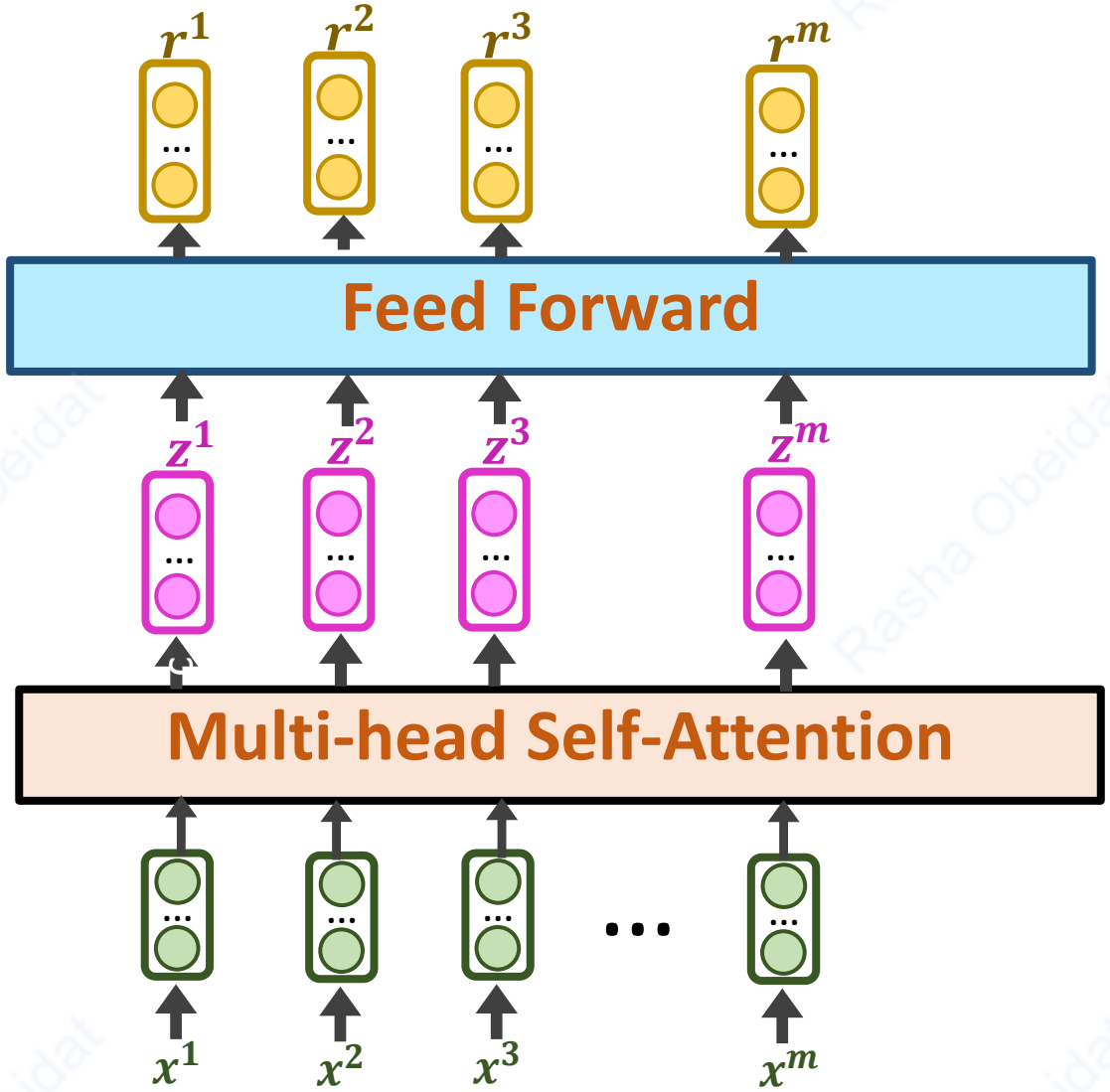
Self-Attention

- **Next Steps:** Transition from Multi-Head Self-Attention to build the complete Transformer Architecture step-by-step



Self-Attention +Feed-Froward NN

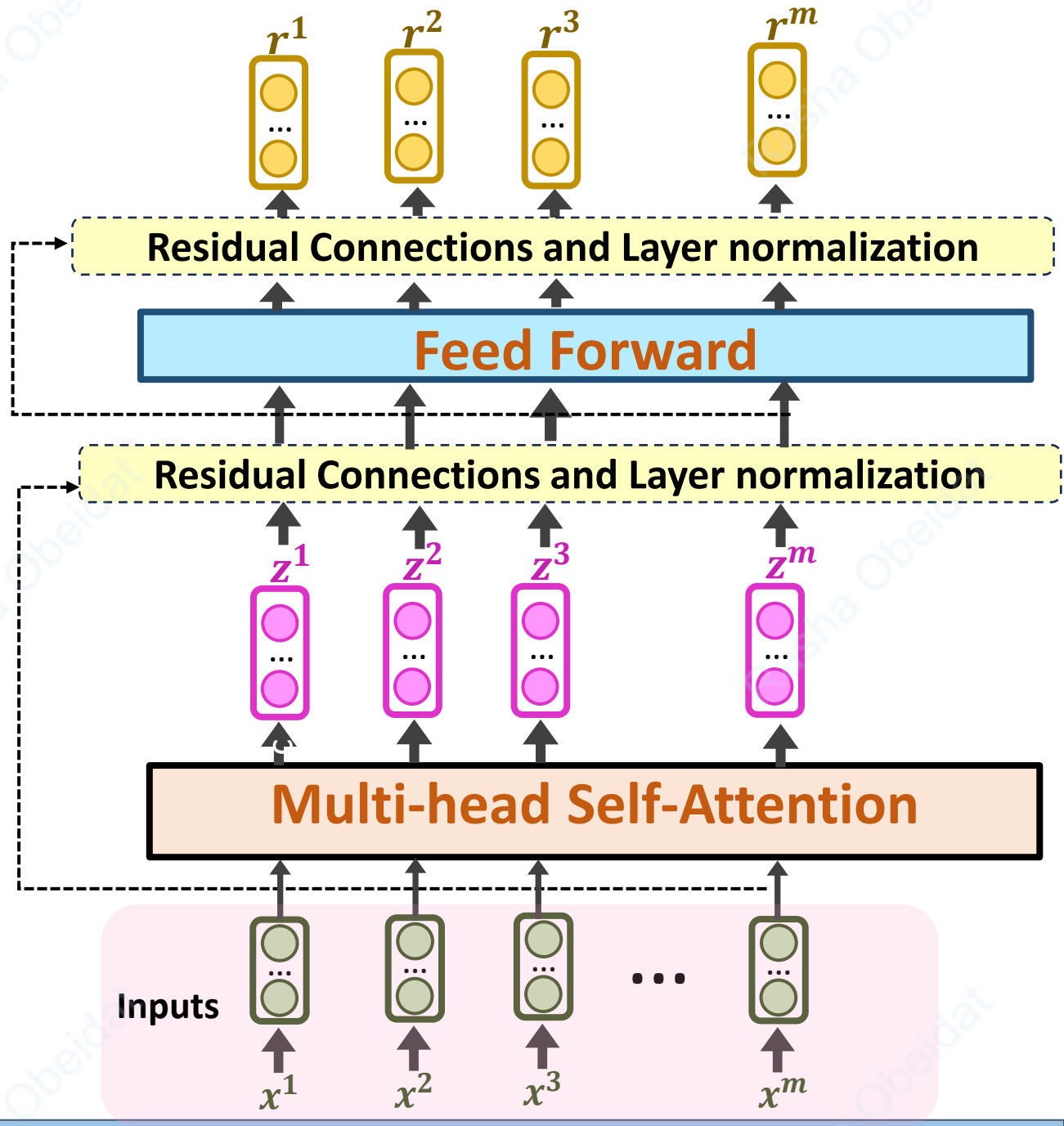
- **Role:** The ***feed-forward*** layer in a Transformer encoder is used to process each token's embedding independently and non-linearly, enhancing its representation after the attention mechanism.
- **Structure:**
  - It consists of two dense (fully connected) layers with an activation function (e.g., ReLU) in between:
    - First layer *expands* the dimension of the embedding.
    - Second layer projects it back to the *original* embedding size.
  - This operation is applied to each token's representation individually.



Residual Connections and Layer Normalization

Residual Connections

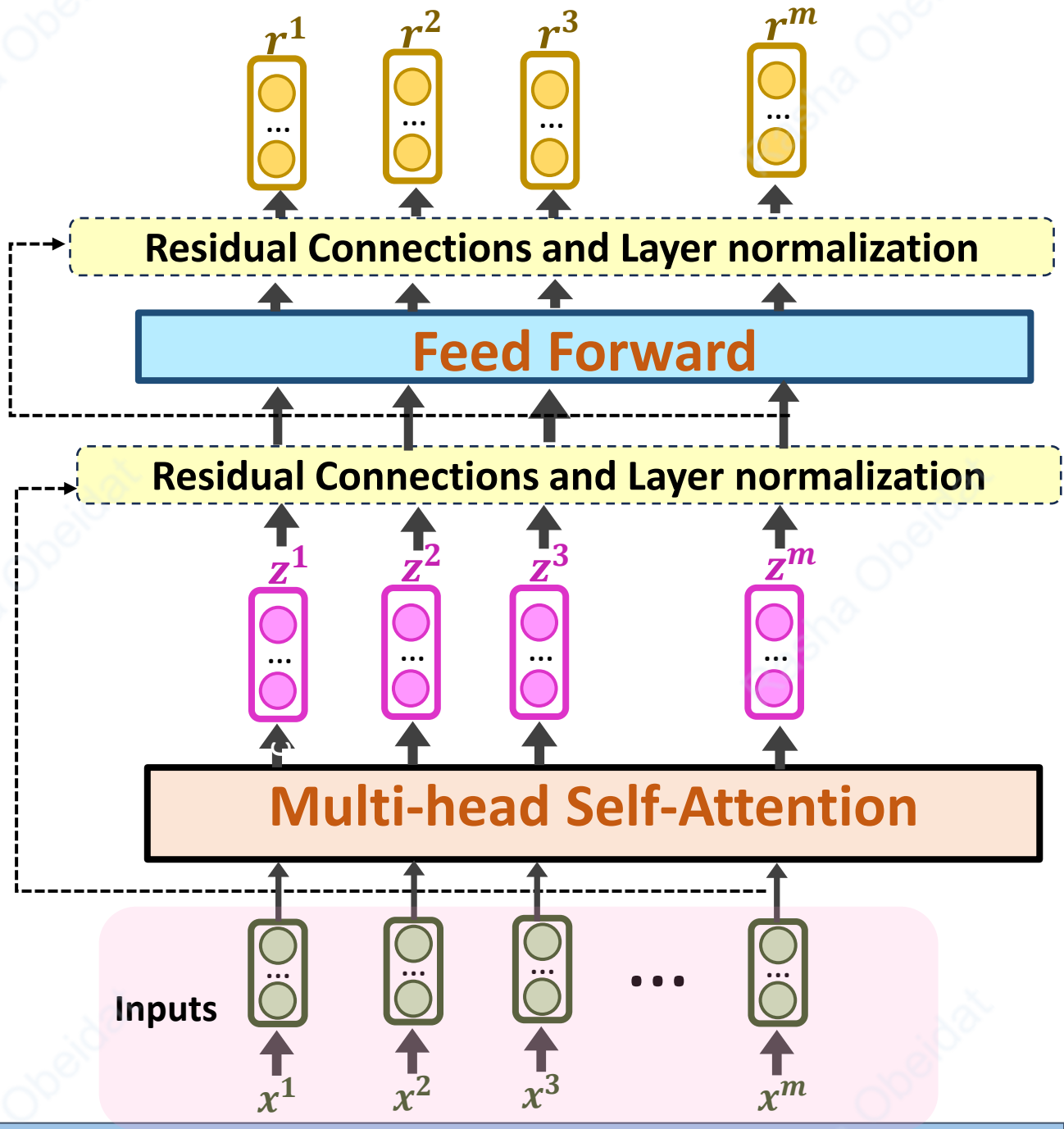
- A **residual connection** (or **skip connection**) simply adds the original input of a layer to its output before passing it to the next layer.



Residual Connections and Layer Normalization

Residual Connections

- A **residual connection** (or **skip connection**) simply adds (***element-wise***) the original input of a layer to its output before passing it to the next layer.
- **The original input to each layer** (like attention or feed-forward) **is** added back to the output after transformation.



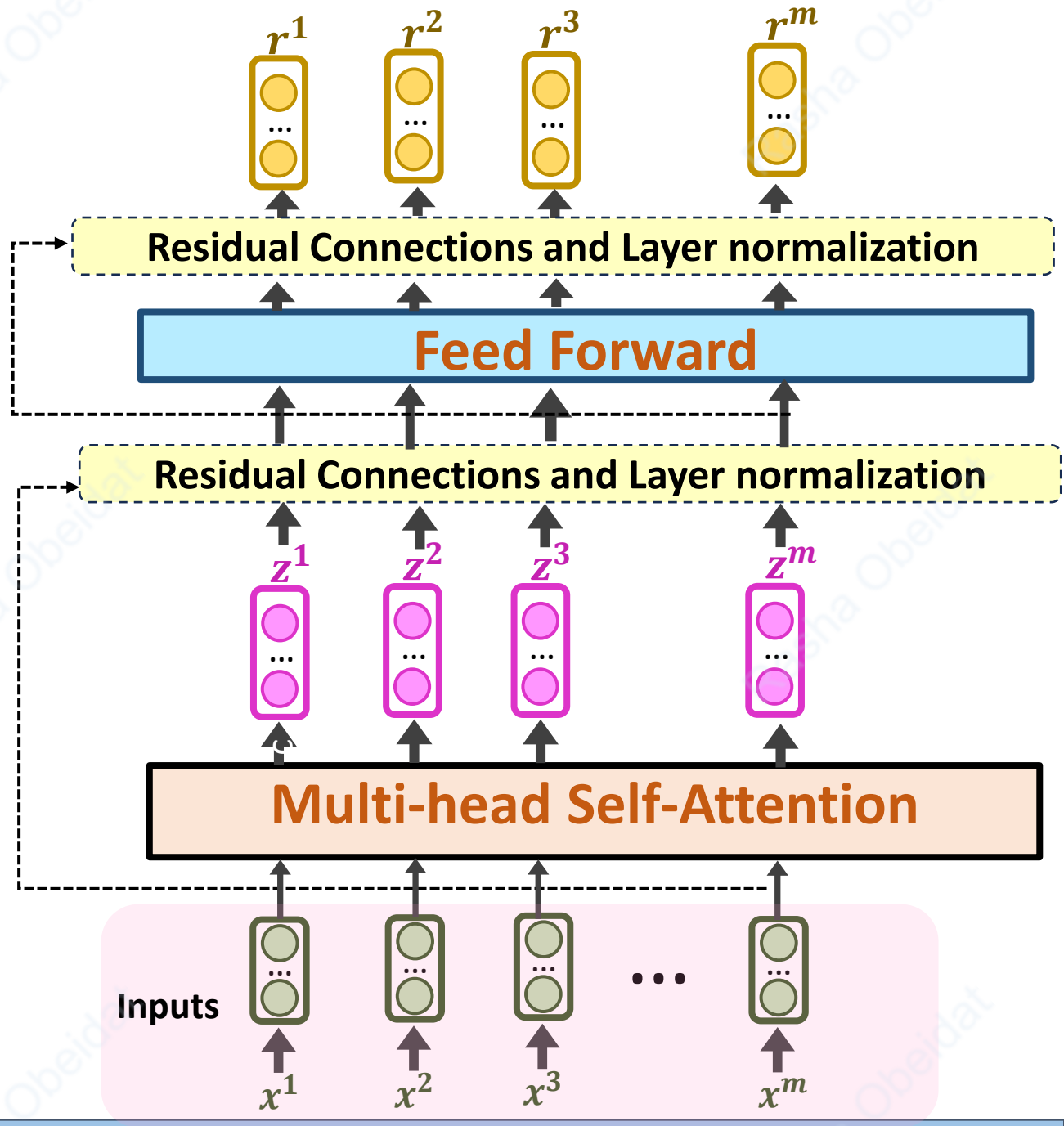
Residual Connections and Layer Normalization

Residual Connections

- A **residual connection** (or **skip connection**) simply adds the original input of a layer to its output before passing it to the next layer.
- **The original input to each layer** (like attention or feed-forward) **is** added back to the output after transformation.

Layer Normalization

- Applied after the residual addition to normalize the summed output



Residual Connections and Layer Normalization

Residual Connections

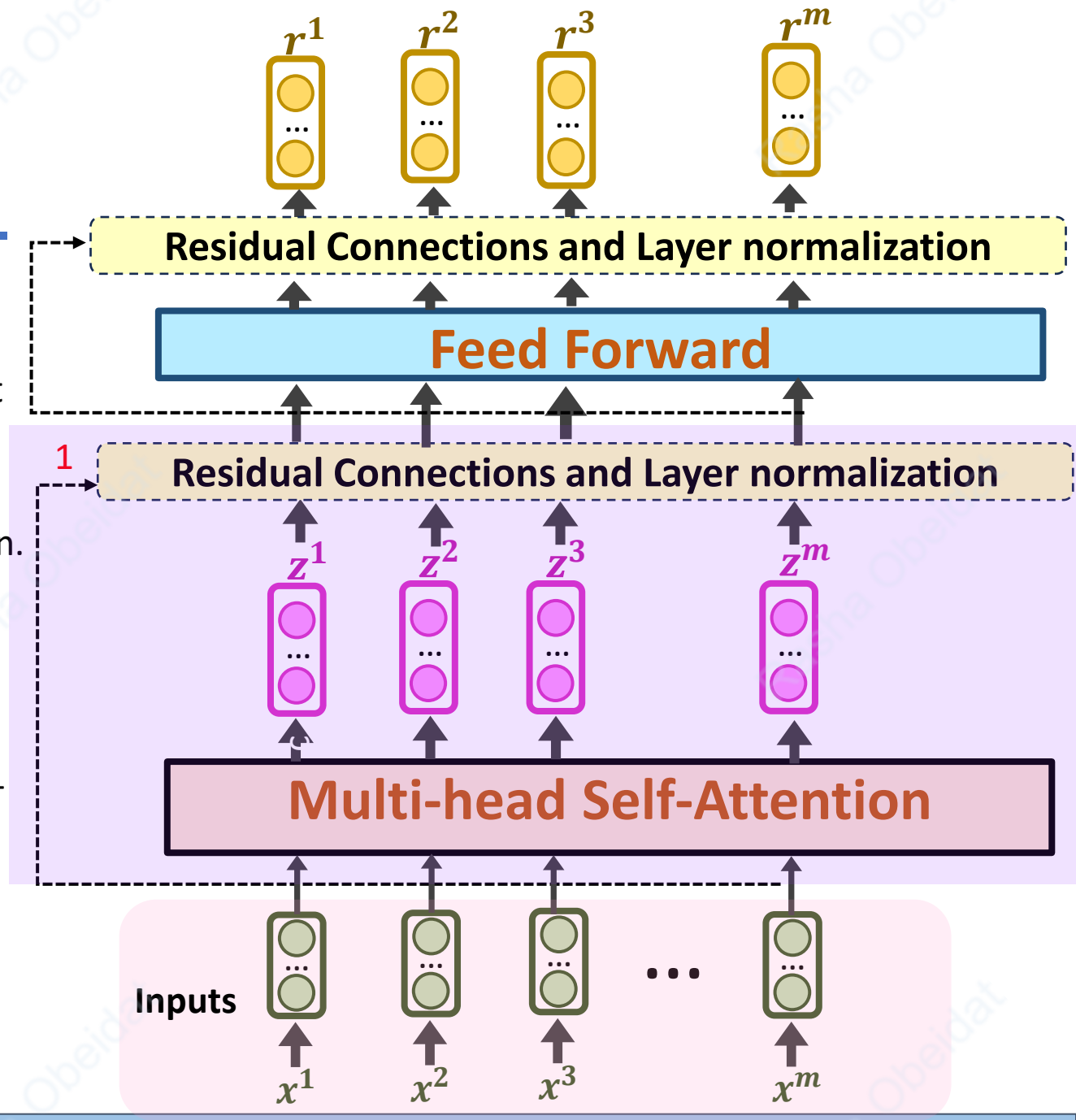
- A **residual connection** (or **skip connection**) simply adds the original input of a layer to its output before passing it to the next layer.
- **The original input to each layer** (like attention or feed-forward) **is** added back to the output after transformation.

Layer Normalization

- Applied after the residual addition to normalize the summed output.

1. In Transformer encoder, the input to the multi-head self-attention x is added to the output of it , then Layer normalization is applied

$$z^i_{resi} = LayerNorm(x^i + z^i)$$



Residual Connections and Layer Normalization

Residual Connections

- A **residual connection** (or **skip connection**) simply adds the original input of a layer to its output before passing it to the next layer.
- **The original input to each layer** (like attention or feed-forward) **is** added back to the output after transformation.

Layer Normalization

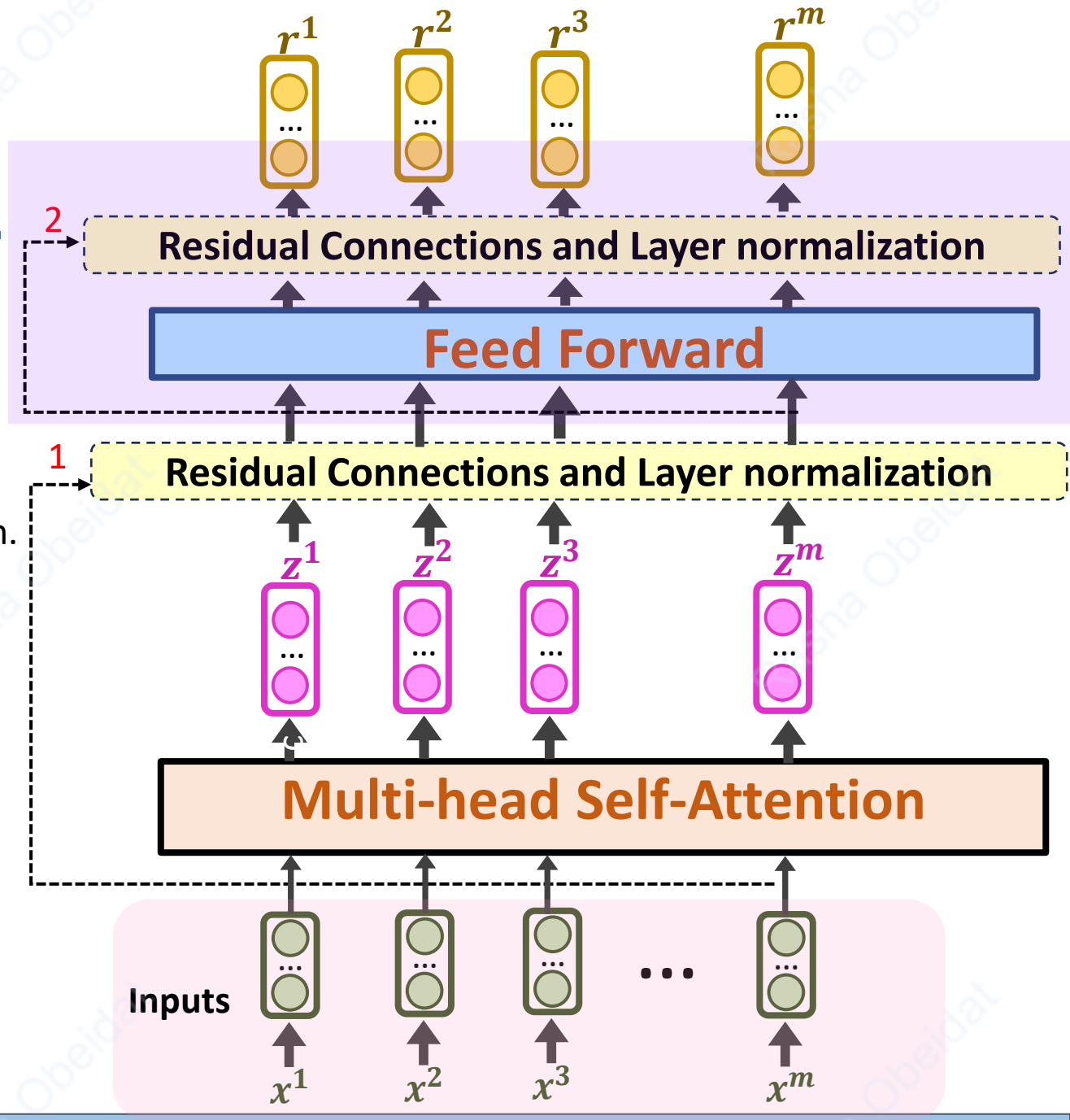
- Applied after the residual addition to normalize the summed output.

1. In Transformer encoder, the input to the multi-head self-attention x is added to the output of it , then Layer normalization is applied

$$z^i_{resi} = LayerNorm(x^i + z^i)$$

2. Then, the the output r^i from the feed-forward block is added to its input

$$r^i_{resi} = LayerNorm(z^i_{resi} + r^i)$$



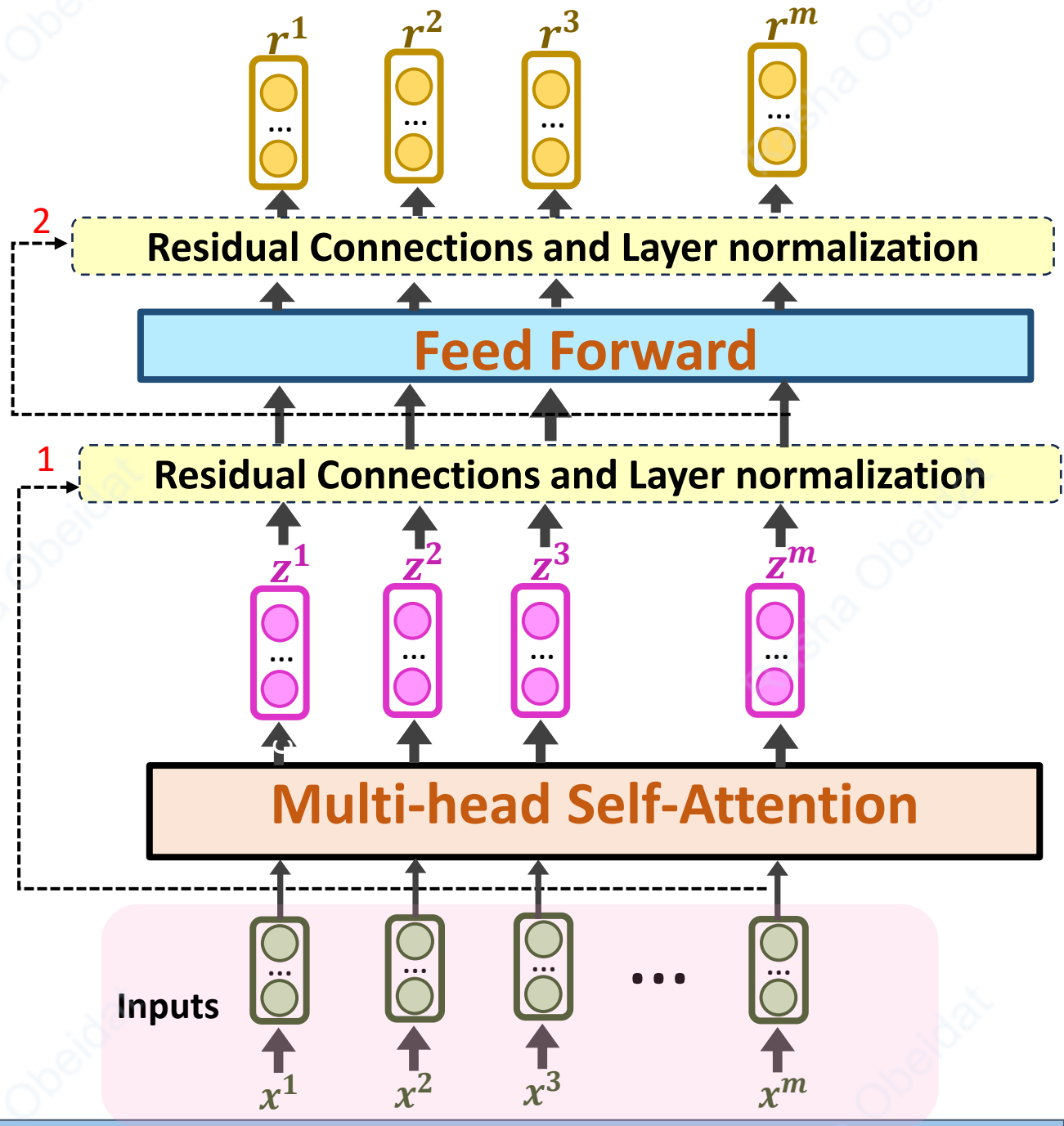
Residual Connections and Layer Normalization

Benefits of Residual Connections

- Help **preserve the original input** , thus preventing loss of information across layers.
- Allow gradients to **flow more easily** during backpropagation → helps train **the model** effectively.

Benefits of Layer Normalization

- Helps reduce the internal covariate shift by keeping the layer's activations well-behaved , thus stabilize and speed up training.



Residual Connections and Layer Normalization

How is layer normalization exactly performed?

- To normalize the vector $x^{\mathrm{i}} \in \mathcal{R}^d$ (e.g., the vectors results from $x^i + z^i$), we perform the following:
- Compute the mean of the vector of the time step i ,

$$\mu^i = \sum_{j=1}^{d} x_j^i$$

- Compute the variance:

$$(\sigma^i)^2 = \sum_{j=1}^{d} (x_j^i - \mu^i)^2$$

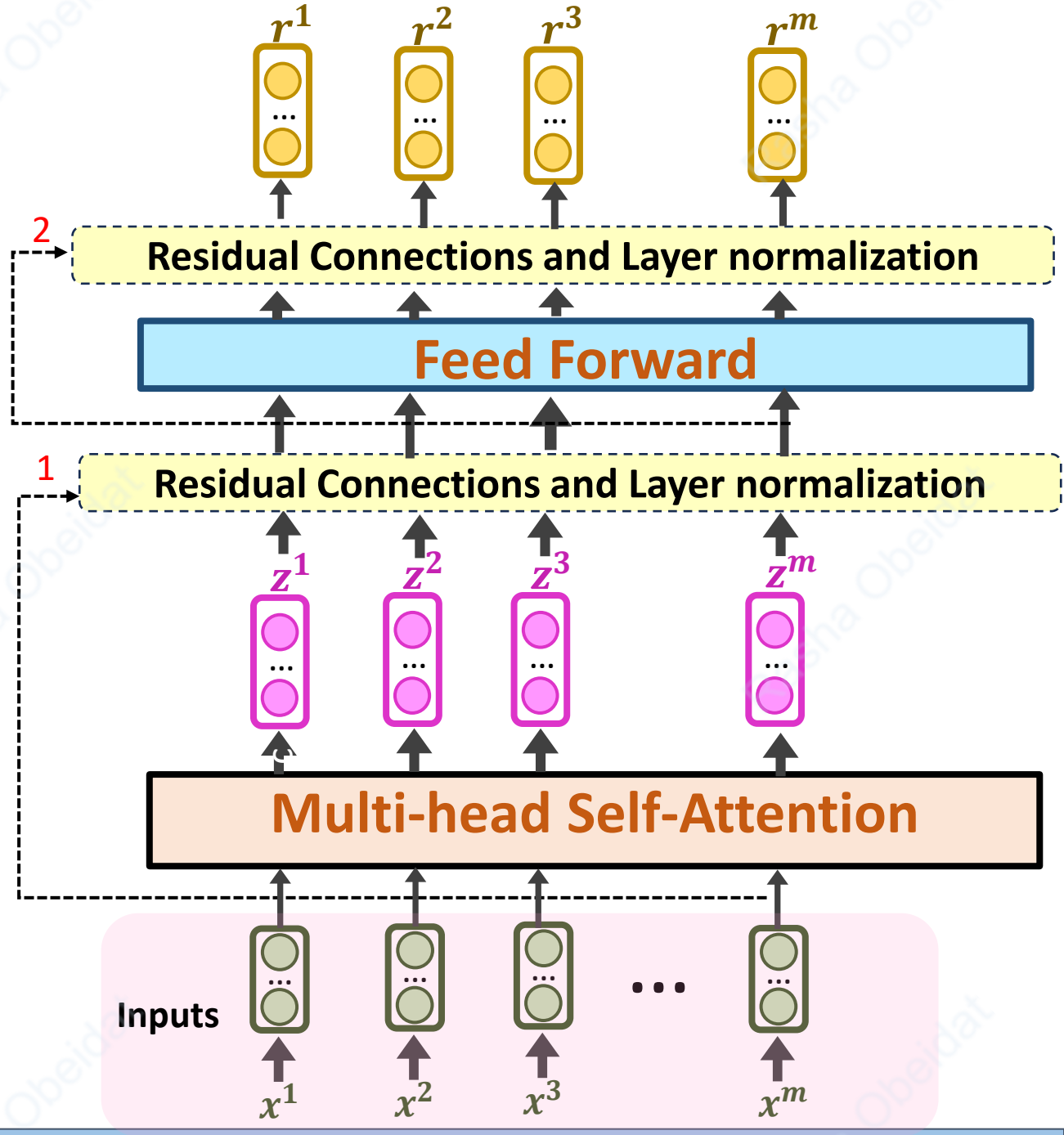
- Normalize each element in x^{i} :

$$\hat{x}_i^j = \frac{x_j^i - \mu^i}{\sqrt{(\sigma^i)^2 + \epsilon}}$$

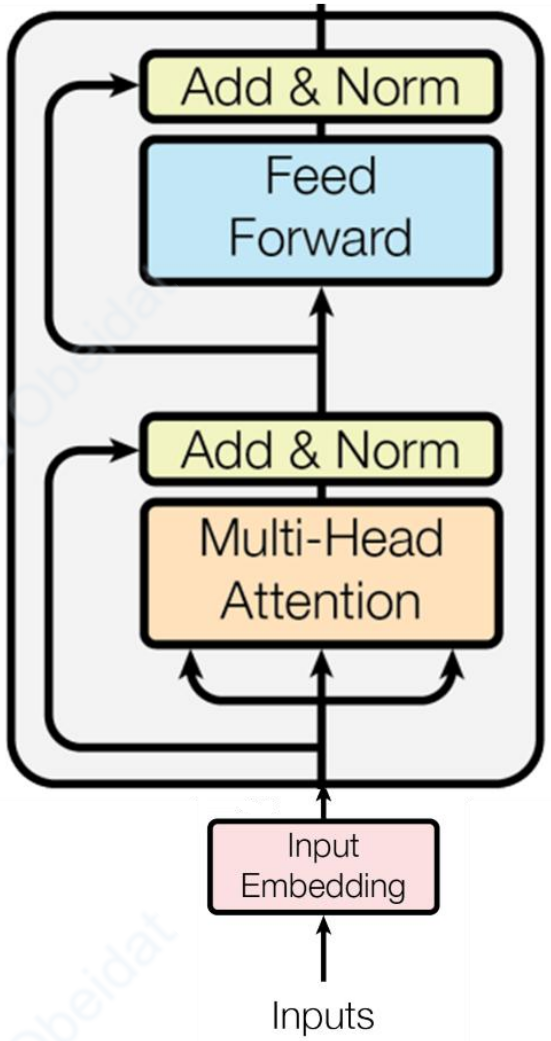
- Apply learned scale and shift:

$$LayerNorm(x^{\mathrm{i}}) = \gamma \hat{x}^i + \beta$$

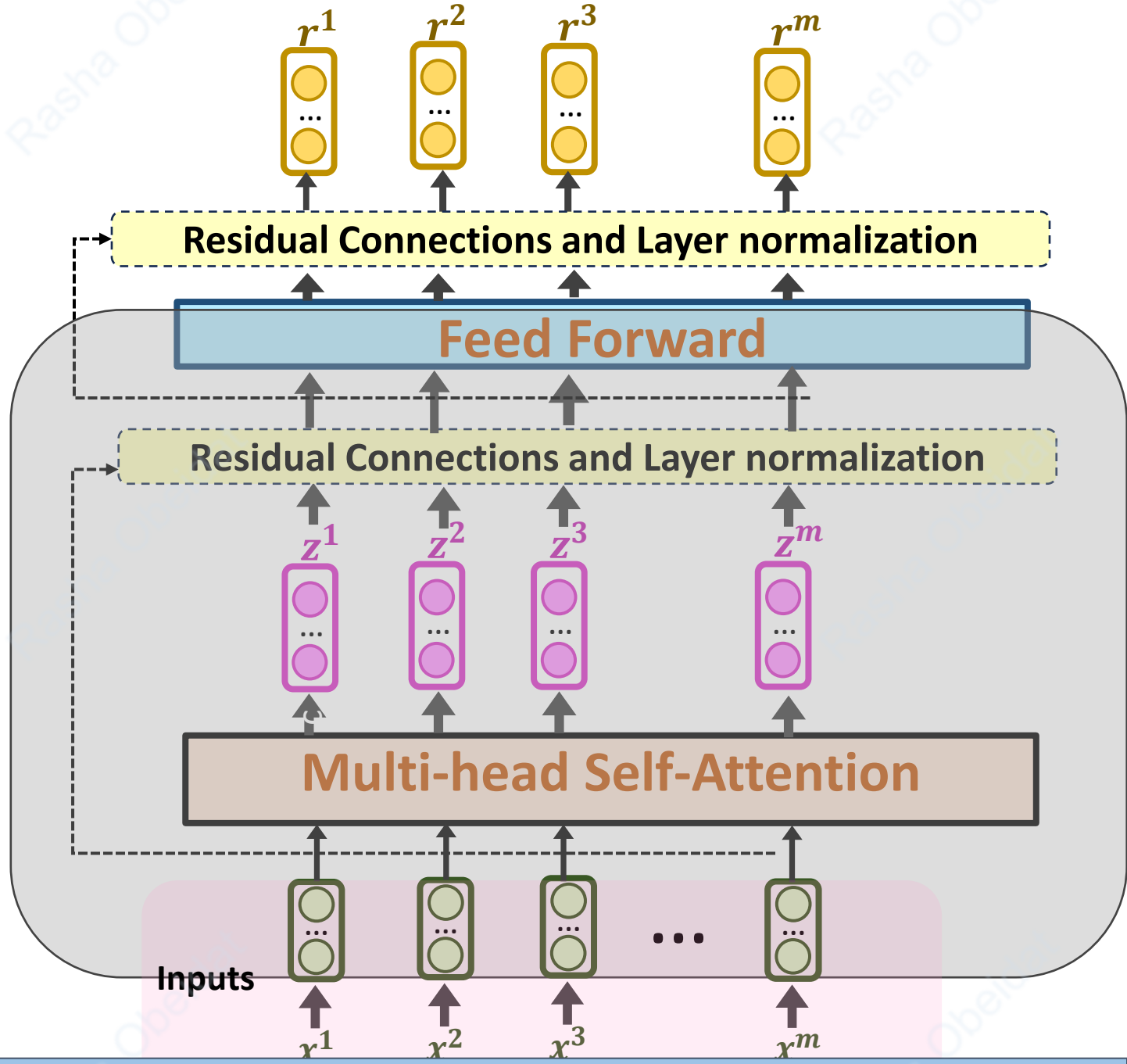
  - γ and β are **trainable parameters** (same size as x^{i}).
  - ϵ is a small constant to prevent division by zero.



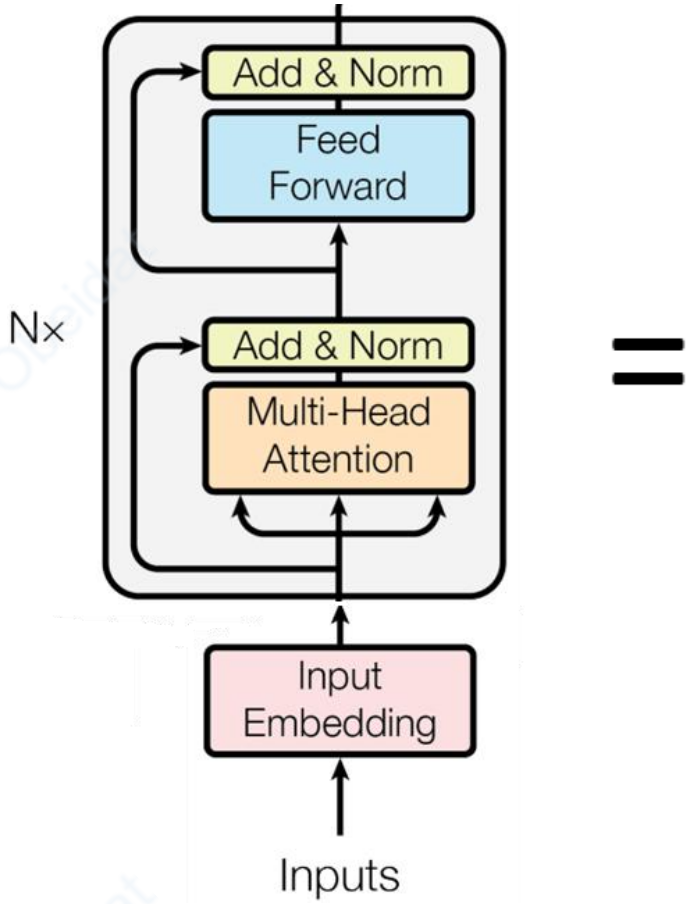
Transformer Encoder



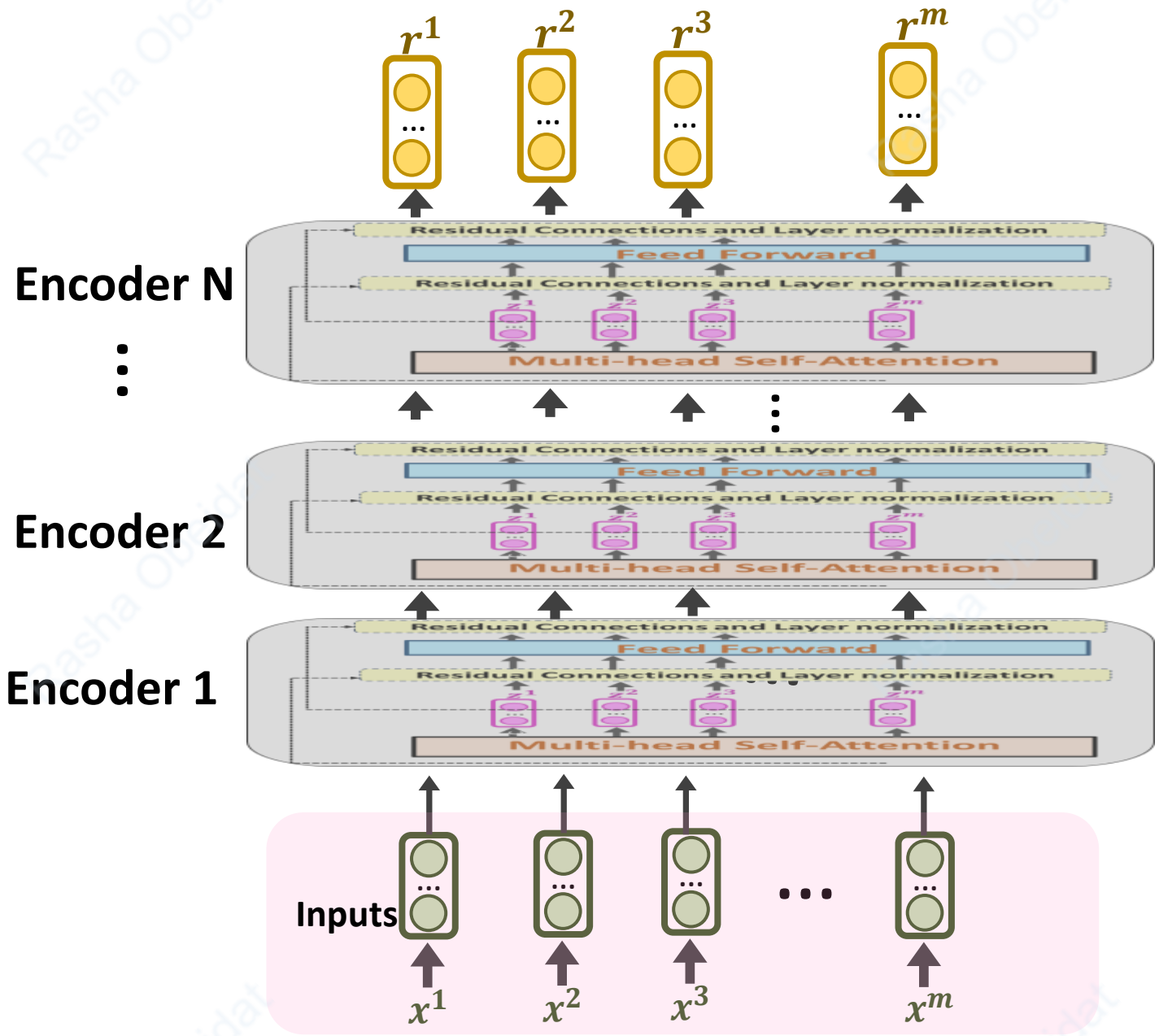
=



Stacking N encoders

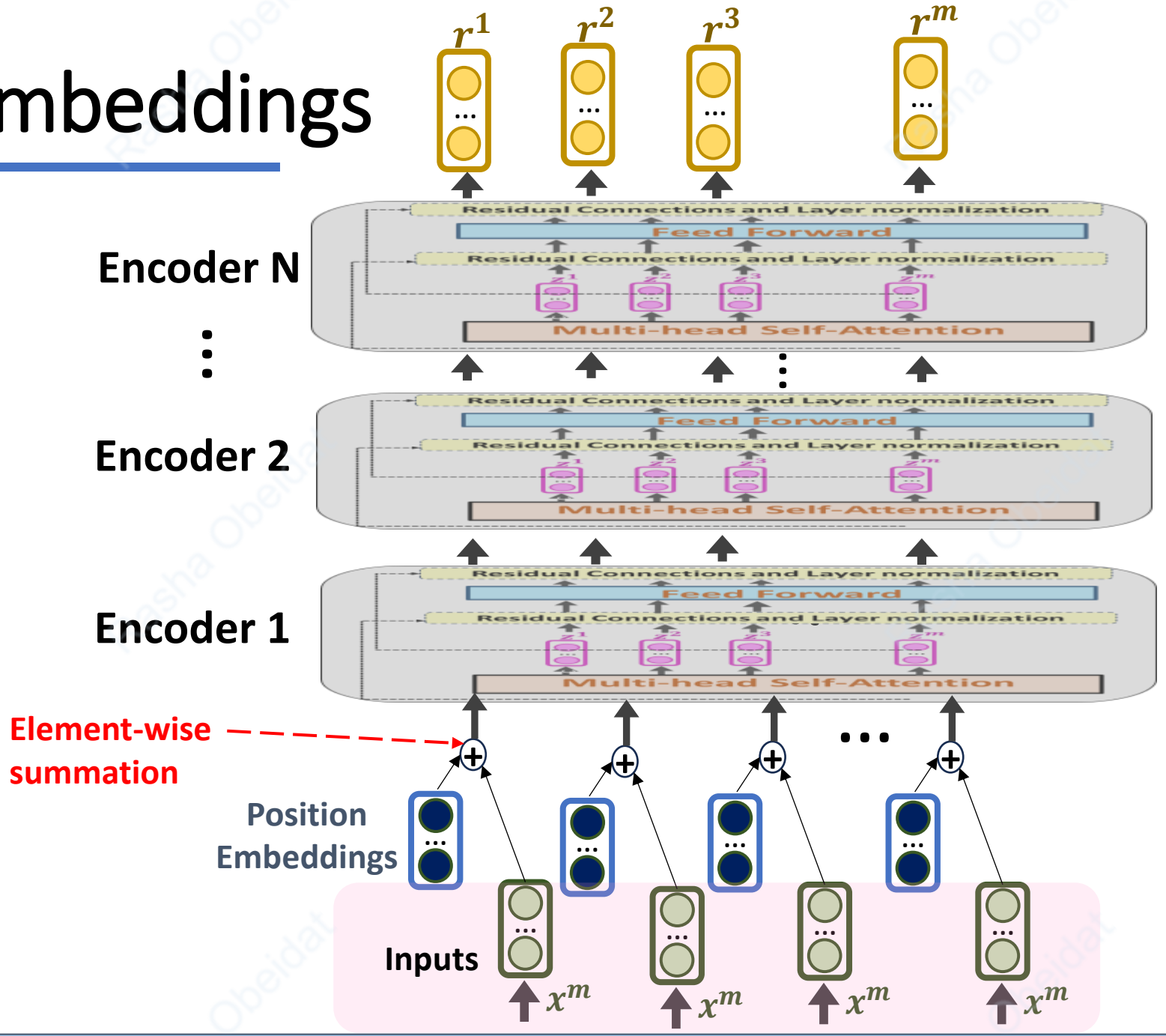


=



Adding Positional Embeddings

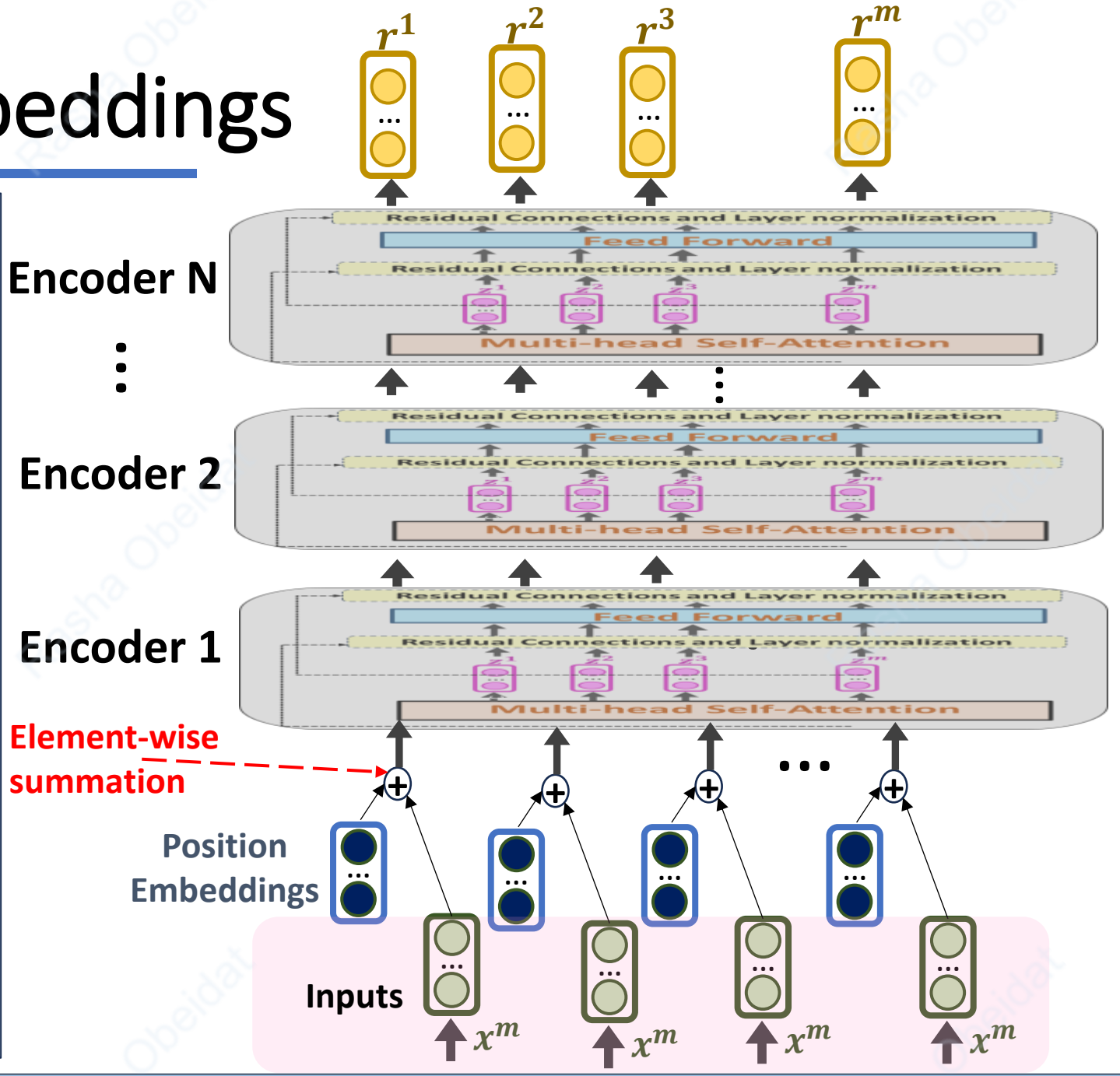
Since the Transformer has **no recurrence or convolution**, it adds **positional information** to each token's embedding using ***positional embeddings***.



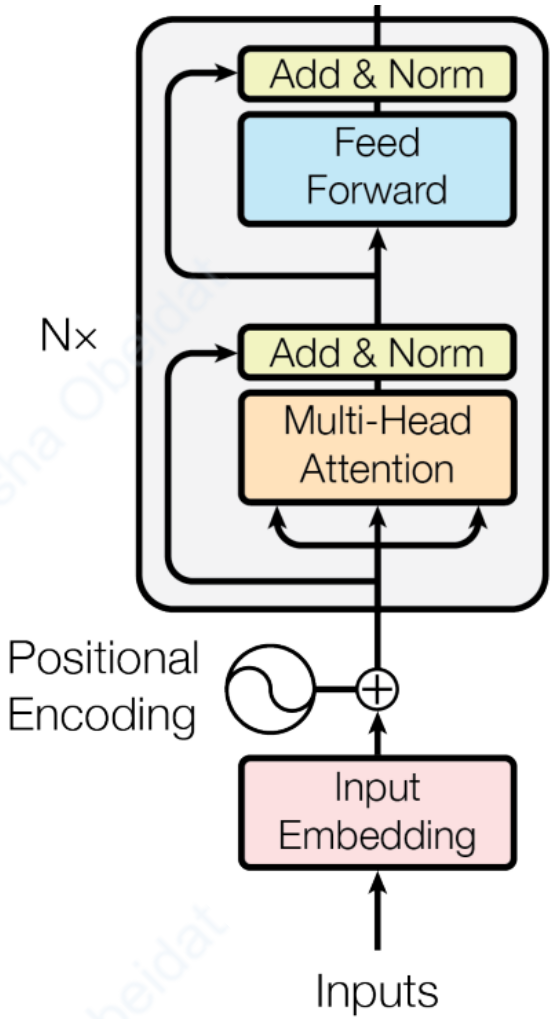
Adding Positional Embeddings

There are two types of positional encodings:

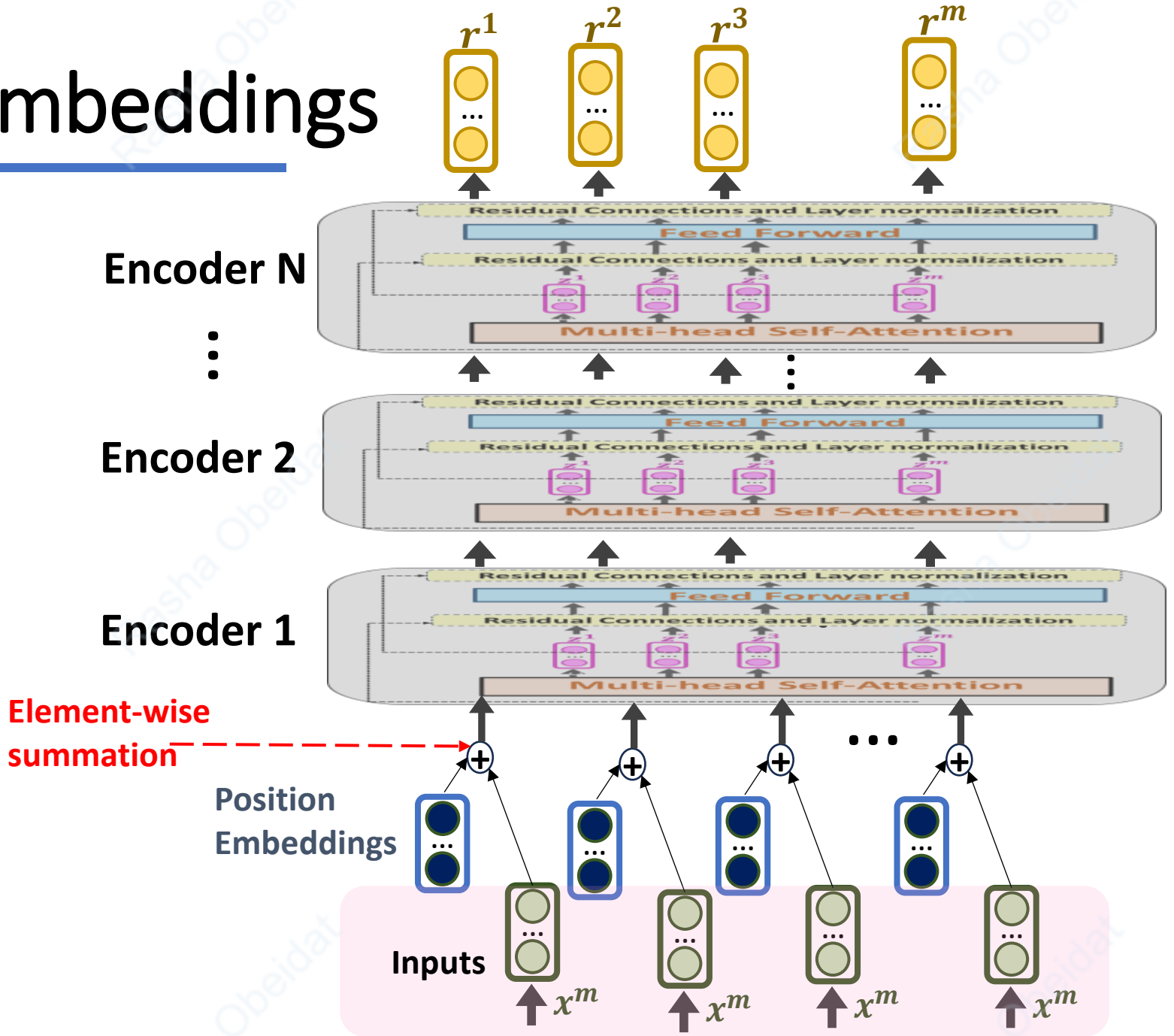
- **Fixed**: use *sine* and *cosine* functions of different frequencies as in the original Transformer paper.
- **Learnable**: position has a dedicated vector that is **initialized randomly** and **updated during training.**
  - The embedding for **each position is fixed during inference**, regardless of the sentence content.
  - For instance, the token at position 1 (second word) in every sentence always receives the **same positional vector**, regardless of what the token is.



Adding Positional Embeddings



=

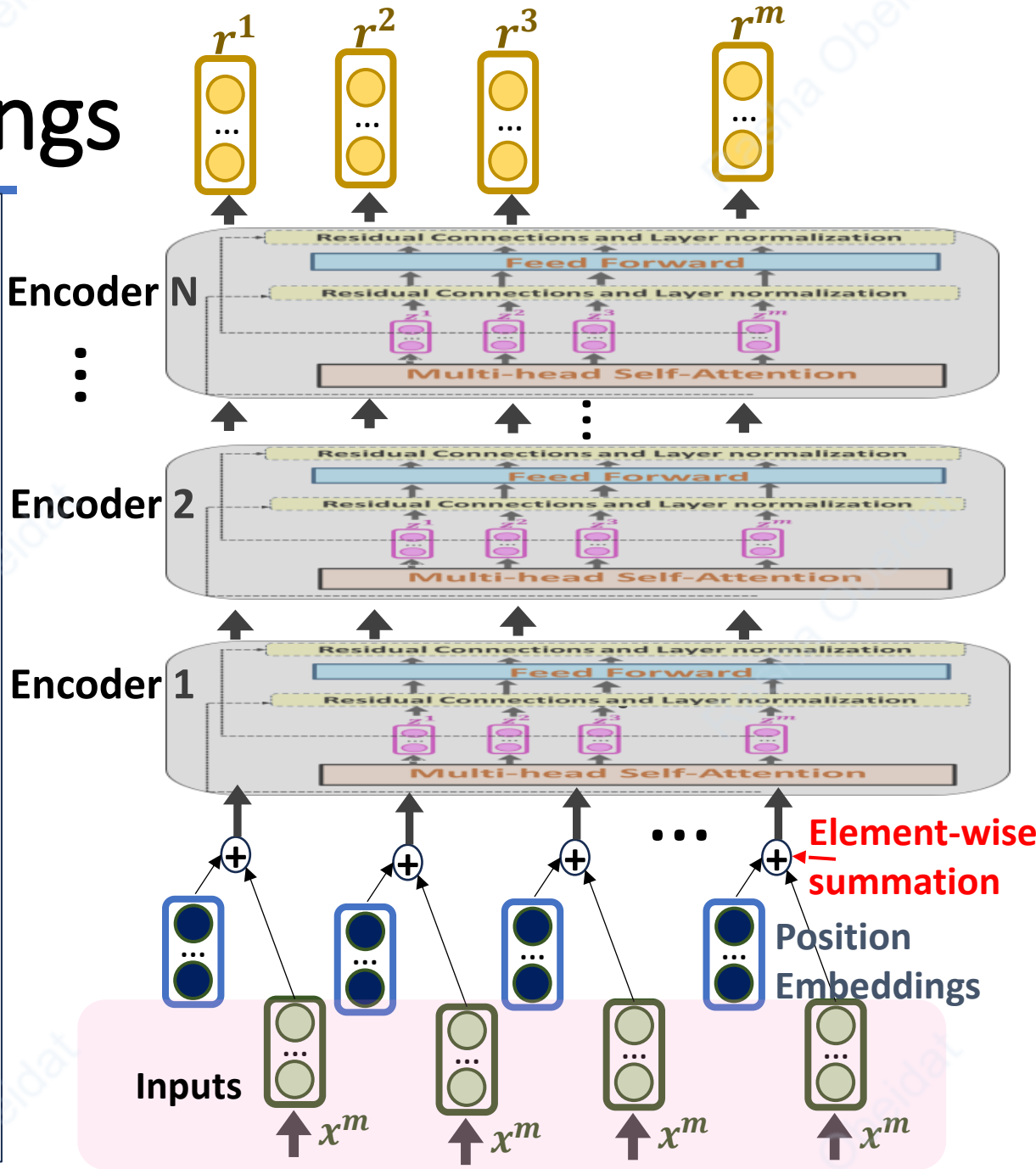


Adding Positional Embeddings

Note: Several vectors and outputs in the Transformer must share the **same dimensionality**, denoted as d_{model}. This consistency ensures that residual connections, layer normalization, and attention mechanisms work properly.

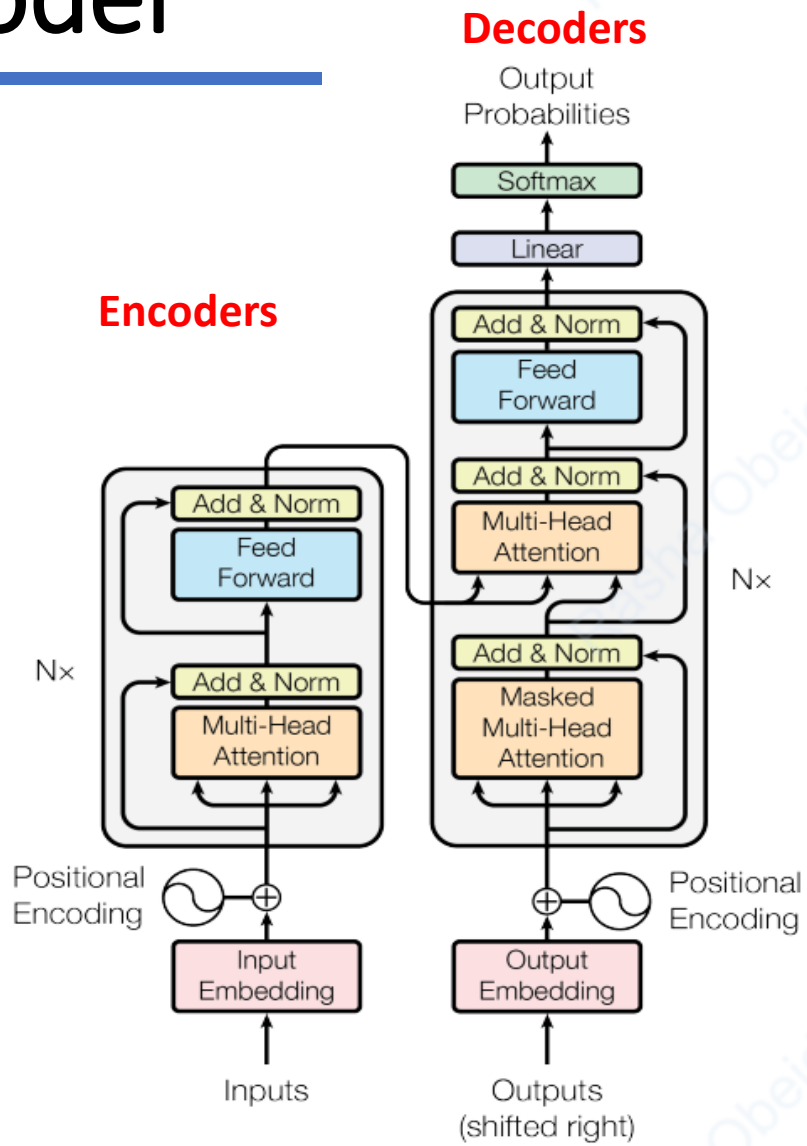
Vectors with the same dimensionality d_{model}

- Word embedding $x^i \in \mathbb{R}^{d_{model}}$
- Positional Embedding $p^i \in \mathbb{R}^{d_{model}}$
- Input to the Encoder Layer: $x^i + p^i \in \mathbb{R}^{d_{model}}$
- Encoder Output (the output of the feed-forward Neural network: $r^i \in \mathbb{R}^{d_{model}}$
- The output of each multi-head attention $z^i \in \mathbb{R}^{d_{model}}$
  - Each single head output has its own dimensionality which equal to d^v for each vector v in that head. Typically, assume that ℓ is the dimensionality of the concatenation of these single head outputs (i.e., $\acute{z}^i$) , the dimensionality of W^O is $\ell \times d_{model}$ that projects the concatenation back to dimensionality d_{model} .



Transformer Encoder and Decoder

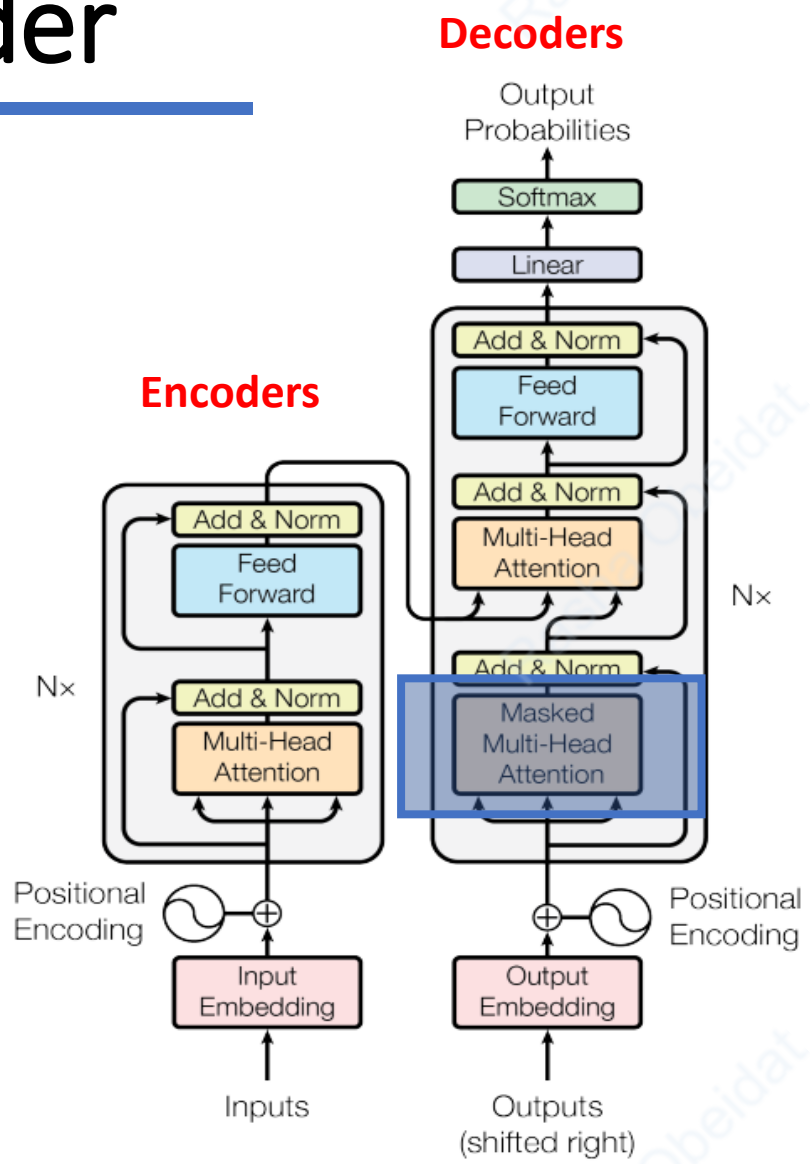
- Transformer model was designed originally for Machine Translation, so it had ***Decoders***, too
- **Transformer Encoders** produce **contextualized embeddings** of each token (e.g., word)
- **Transformer Decoders** generate new sequences of tokens



Transformer Encoder and Decoder

- Transformer **Decoder** is similar to the **Encoder**, except:

1. The decoder has a Masked Multi-Head Self-Attention.

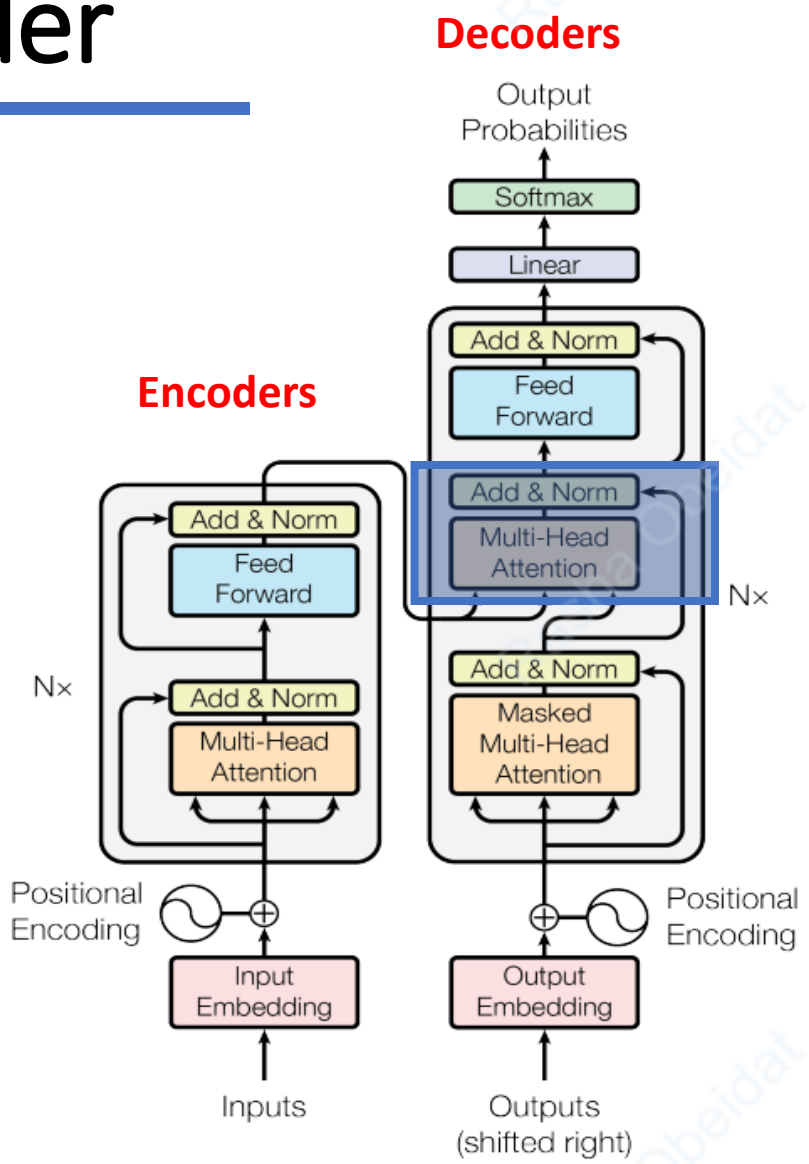


Transformer Encoder and Decoder

- Transformer **Decoder** is similar to the **Encoder**, except:

1. The decoder has a Masked Multi-Head Self-Attention .

2. Uses additional Encoder-Decoder Attention.



Decoder's Masked Multi-Head Self-Attention

- It's a self-attention mechanism that allows each position to **only attend to itself and earlier positions**, not to future tokens — ensuring ***causality*** during generation.
- **Why Do We Need Masking?**

  In tasks like translation or (text generation in general) :
  - The Decoder should generate tokens **one by one**, using only previous outputs.
  - At time step t, the model must **not look ahead** at position $t+1, t+2, \ldots m$. In other words
  - Masking prevents the decoder from "***cheating***" by looking at the answer it hasn't generated yet, thus, preventing **information leakage** from future tokens.
  - ensure that the decoder operates in an **auto-regressive manner**, just like during inference without accessing "future" information making Makes training and inference consistent.
  - Enables **controlled generation** in tasks like translation, summarization, and language modeling.

Decoder's Masked Multi-Head Self-Attention

Remember: During training, we have access to the **ground-truth translation**, but we still follow the idea that the **decoder generates the output one word at a time**, relying on:

- The encoder's representation of the input, and
- The words that have already been generated so far. The first self-attention head in the **Transformer decoder** is **masked** to ensure that the decoder operates in an **auto-regressive manner**, just like during inference without accessing "future" information.

Decoder's Masked Multi-Head Self-Attention

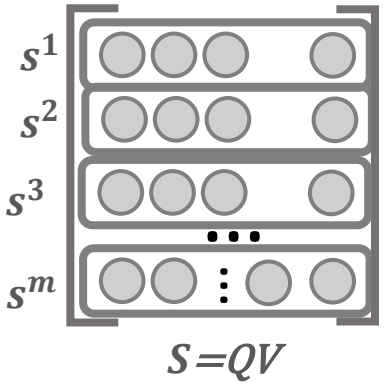
How masking is performed?

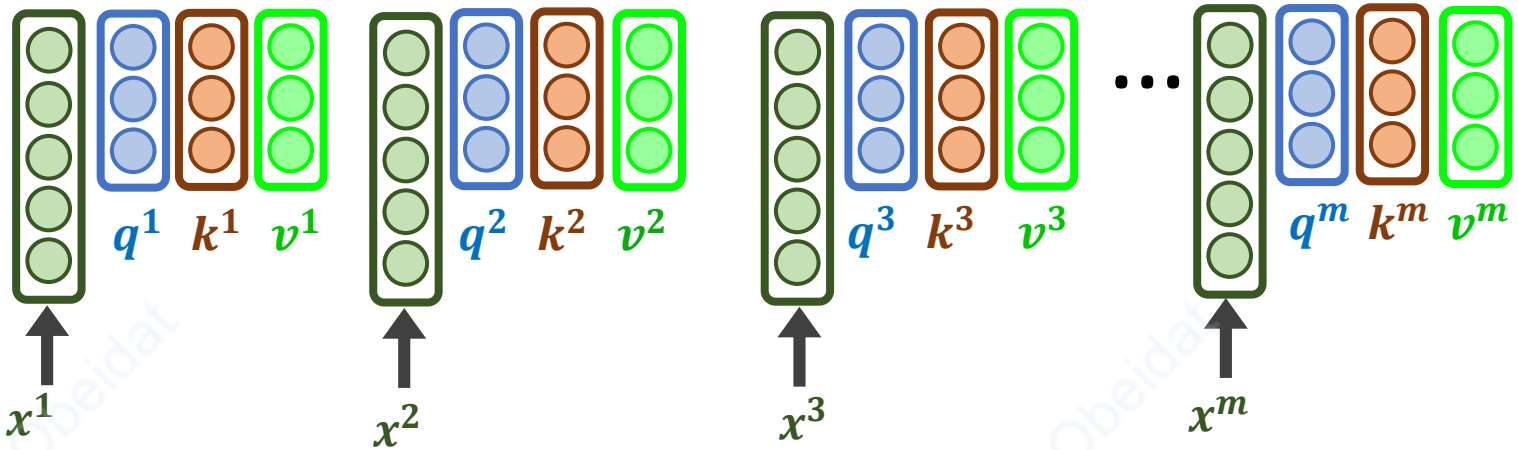
- By applying mask before Softmax (usually an upper-triangular mask matrix).

- A **mask matrix** $M \in \mathbb{R}^{n \times n}$ is created, where:
  - $M^{i,j} = 0, \quad if\ j \leq i$ (allowed)
  - $M^{i,j} = -\infty, \quad if\ j > i$ (masked)
  - Apply the matrix M on $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d^k}} + \text{M}\right)V$
  - **For example**, in sequence of 4 tokens,
    - At time step $t = 1$ the model can attend to the token x^1 and the corresponding row in the matrix M (the first row) is $[0, -\infty, -\infty, -\infty]$ which means [**allow, mask, mask, mask**].
    - At time step $t = 2$ the model can attend to the token x^2 and the corresponding row in the matrix M (the second row) is $[0, 0, -\infty, -\infty]$ which means [**allow, allow, mask, mask**].
    - At time step $t = 3$ the model can attend to the token x^3 and the corresponding row in the matrix M (the second row) is $[0, 0, 0, \ -\infty]$ which means [**allow, allow, allow, mask**].
    - At time step $t = 3$ the model can attend to the token x^3 and the corresponding row in the matrix M (the second row) is $[0, 0, 0, 0]$ which means [**allow, allow, allow, allow**].

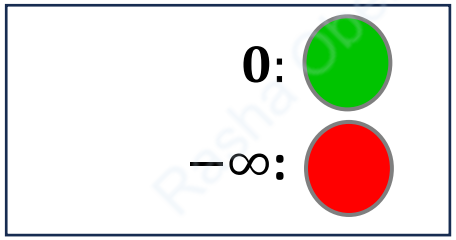
Self-Attention

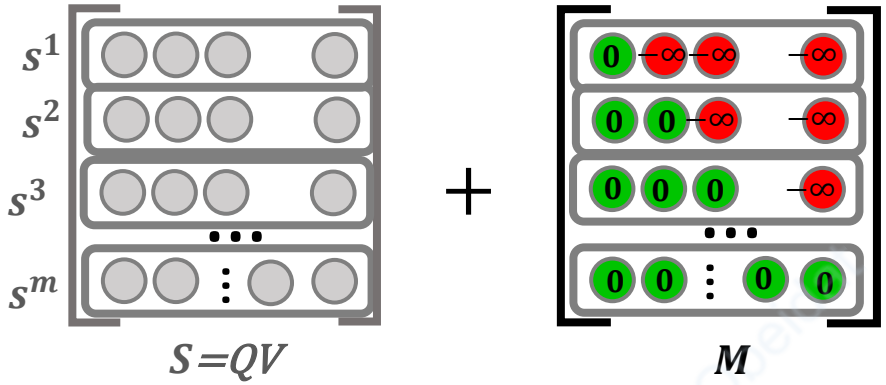
$$s^1 = [s^{1,1}, s^{1,2}, s^{1,3}, \ldots, s^{1,m}]^T$$
$$s^2 = [s^{2,1}, s^{2,2}, s^{2,3}, \ldots, s^{2,m}]^T$$
$$s^3 = [s^{3,1}, s^{3,2}, s^{3,3}, \ldots, s^{3,m}]^T$$
$$\ldots$$
$$s^m = [s^{m,1}, s^{m,2}, s^{m,3}, \ldots, s^{m,m}]^T$$

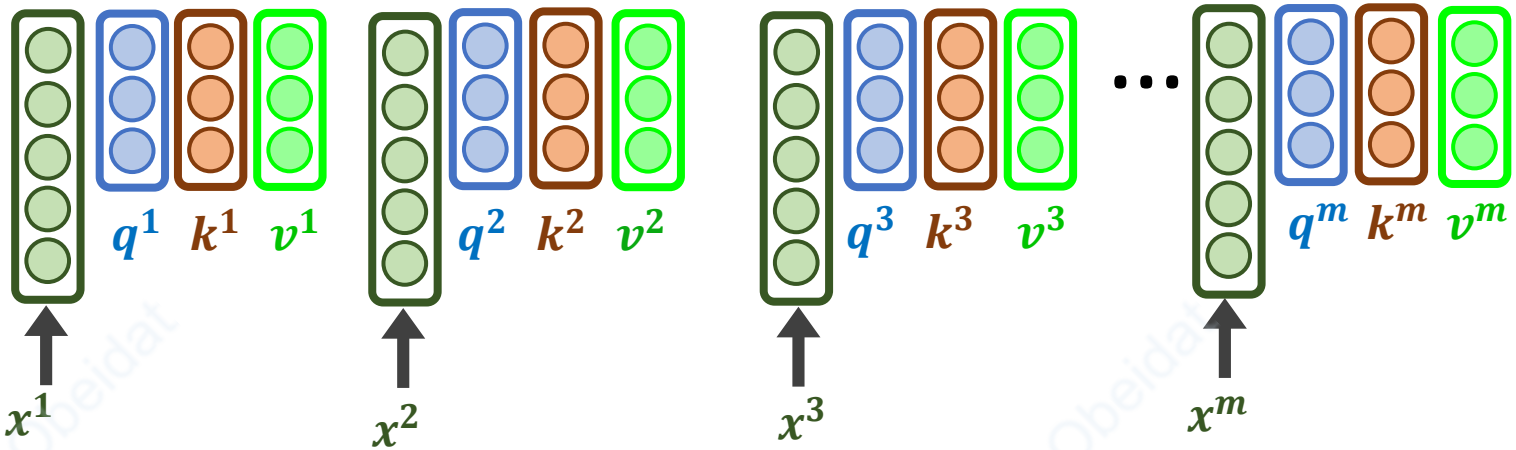




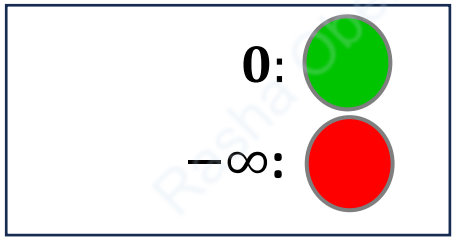
Masked Self-Attention

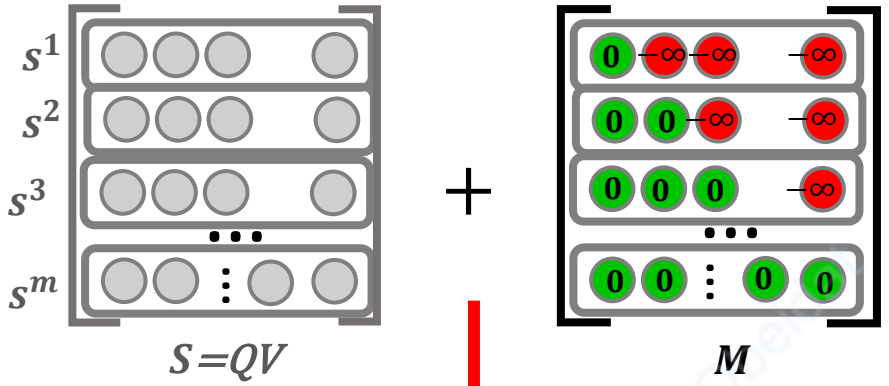


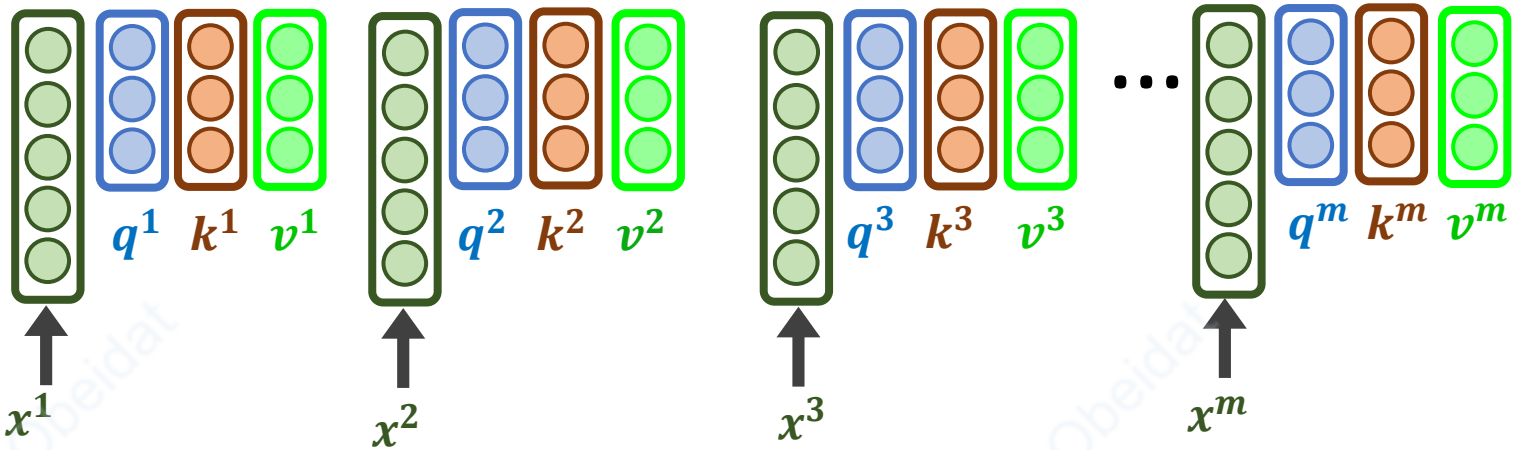


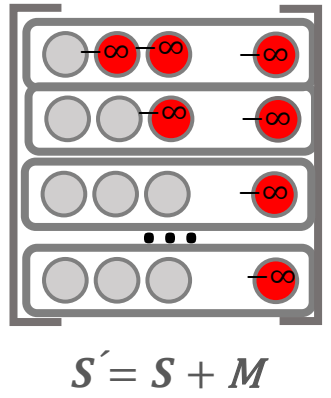


Masked Self-Attention





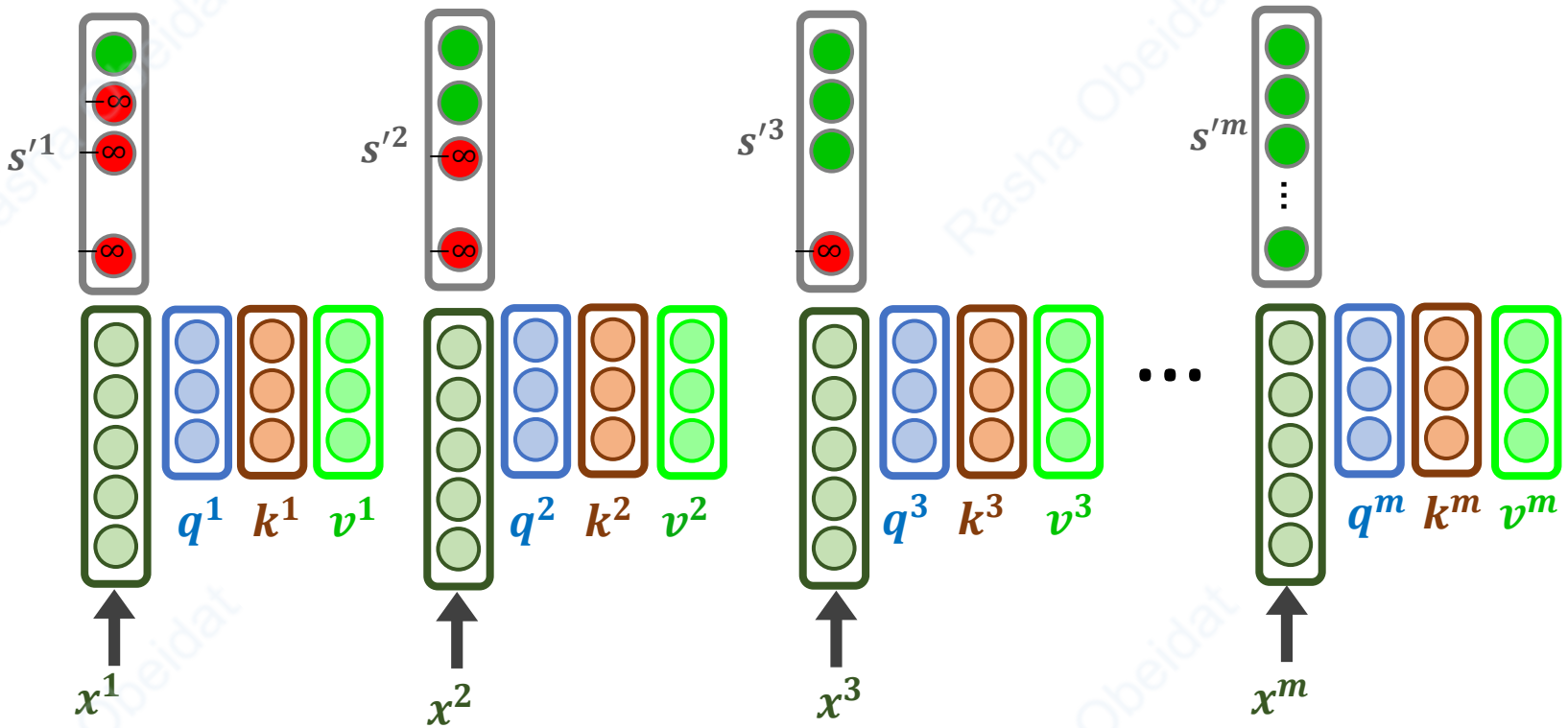


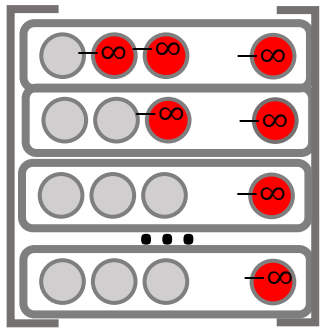


Masked Self-Attention

- $\boldsymbol{S}' = \boldsymbol{S} + \boldsymbol{M}$ will be used to compute the attention instead of using $\boldsymbol{S}$:

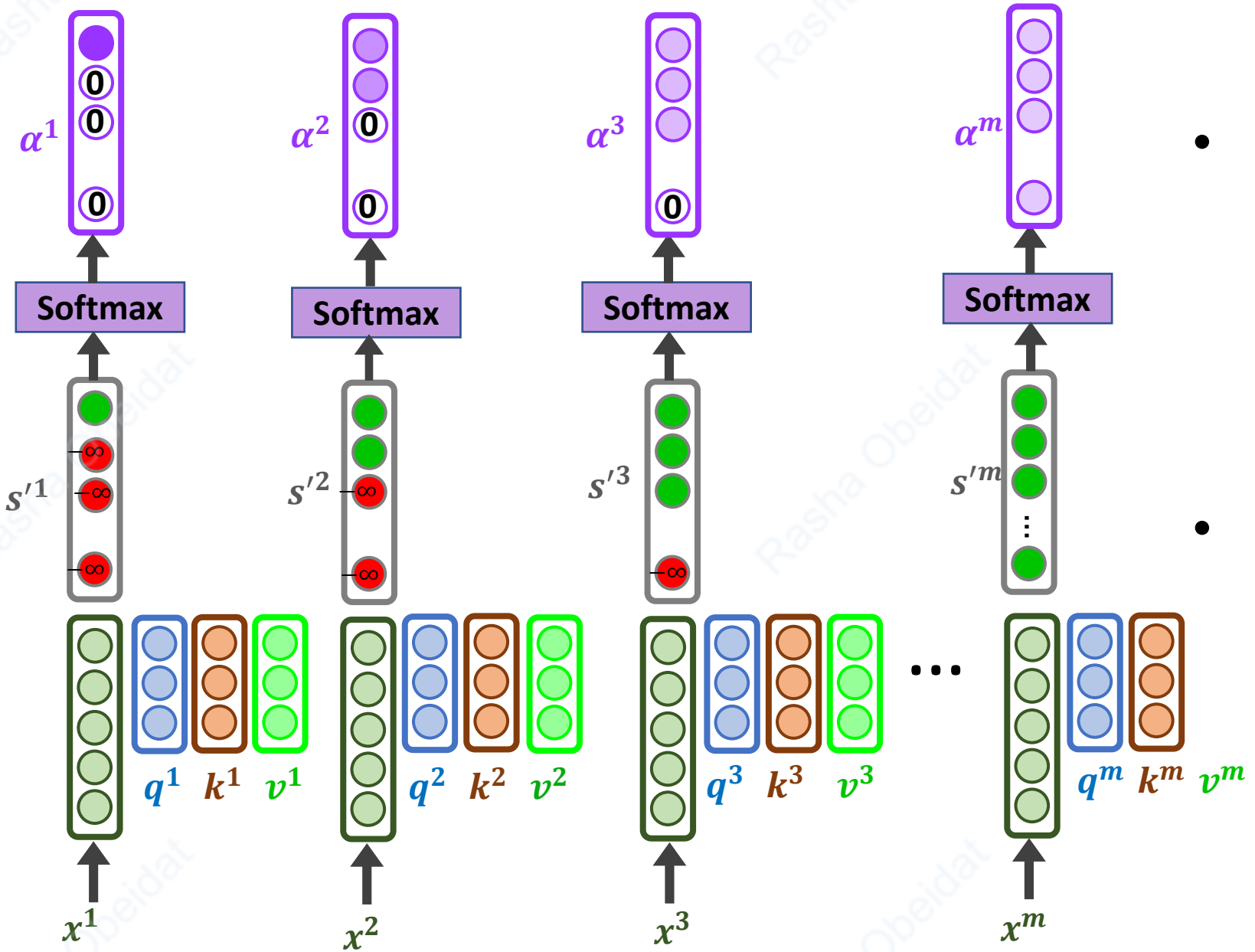
$$\alpha^i = \text{Softmax}\left(\frac{\acute{s}^i}{\sqrt{d^k}}\right)$$





$S' = S + M$

Masked Self-Attention



- $\boldsymbol{S}' = \boldsymbol{S} + \boldsymbol{M}$ will be used to compute the attention instead of using $\boldsymbol{S}$:

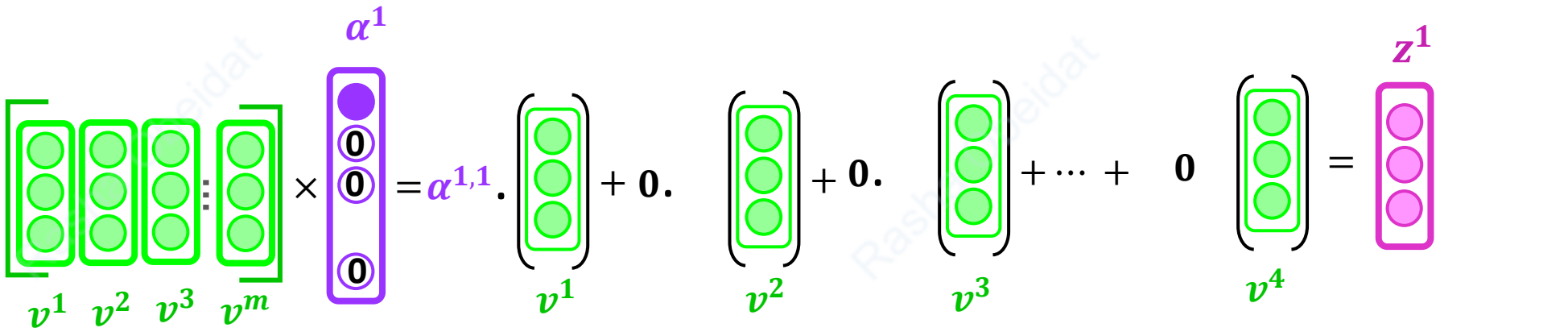
$$\alpha^i = \text{Softmax}\left(\frac{\acute{s}^i}{\sqrt{d^k}}\right)$$

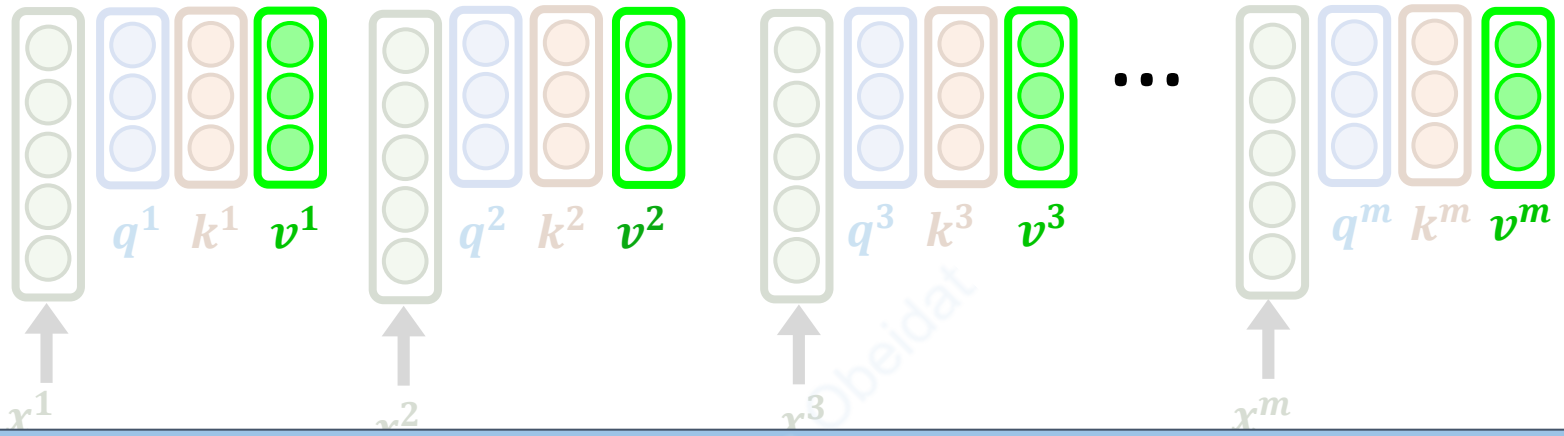
- The softmax will assign near-zero probability to masked positions.

Masked Self-Attention

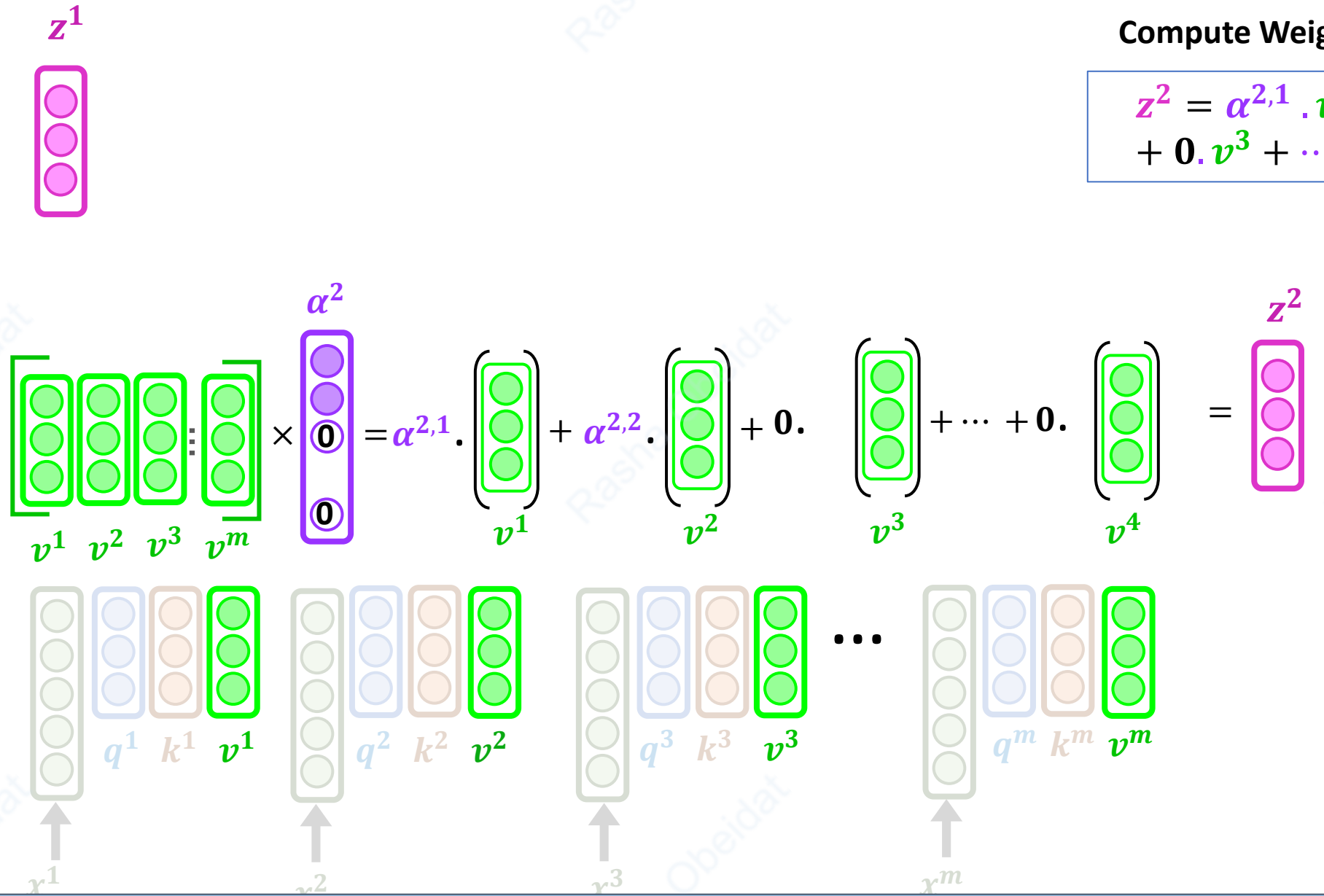
Compute Weighted Sum

$$z^1 = \alpha^{1,1} \cdot v^1 + 0 \cdot v^2 + 0 \cdot v^3 + \cdots + 0 \cdot v^m$$

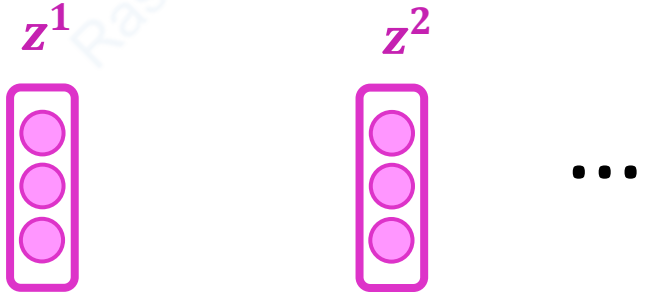




Masked Self-Attention

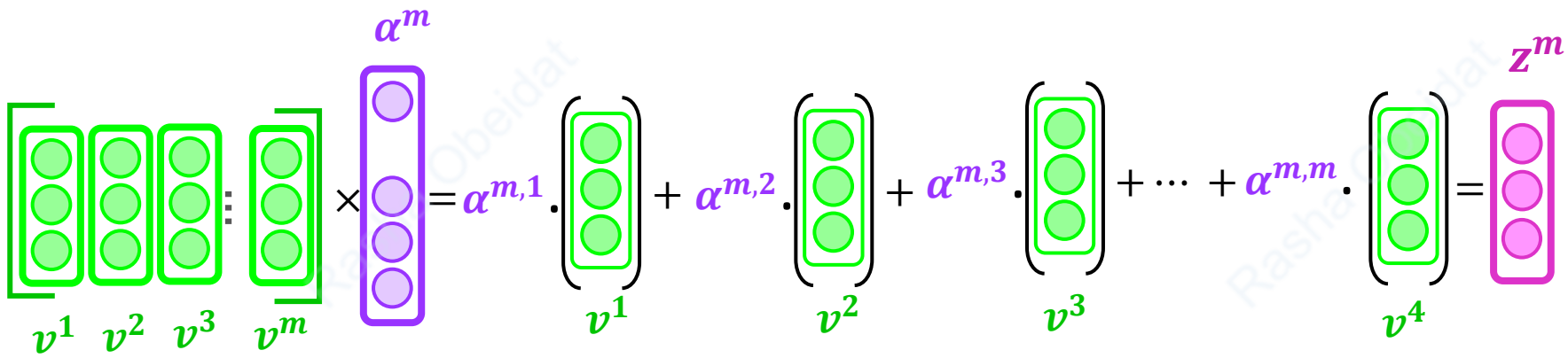


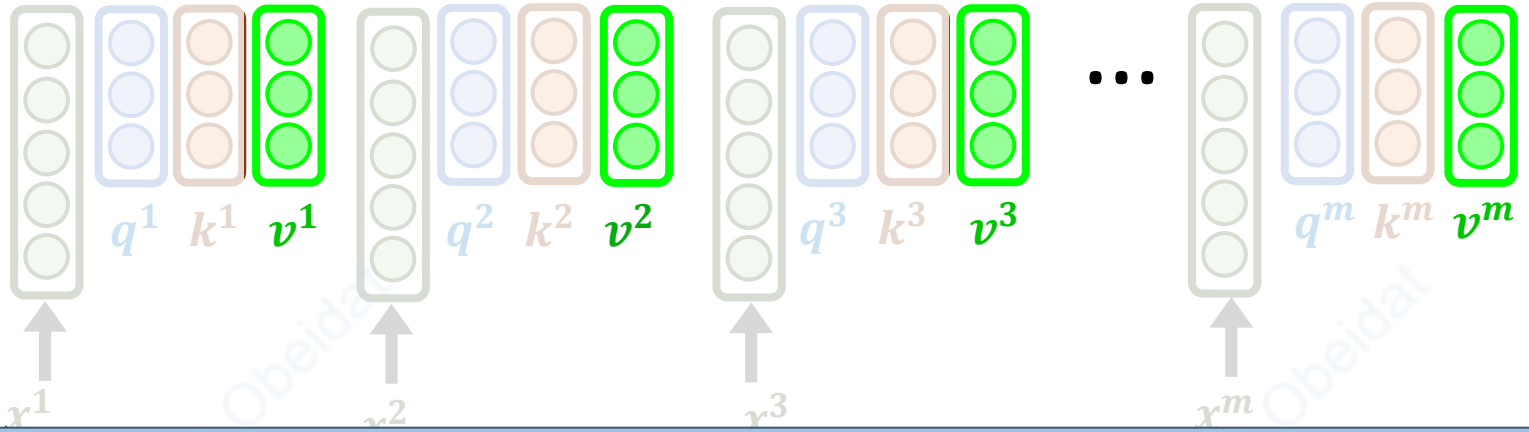
Masked Self-Attention



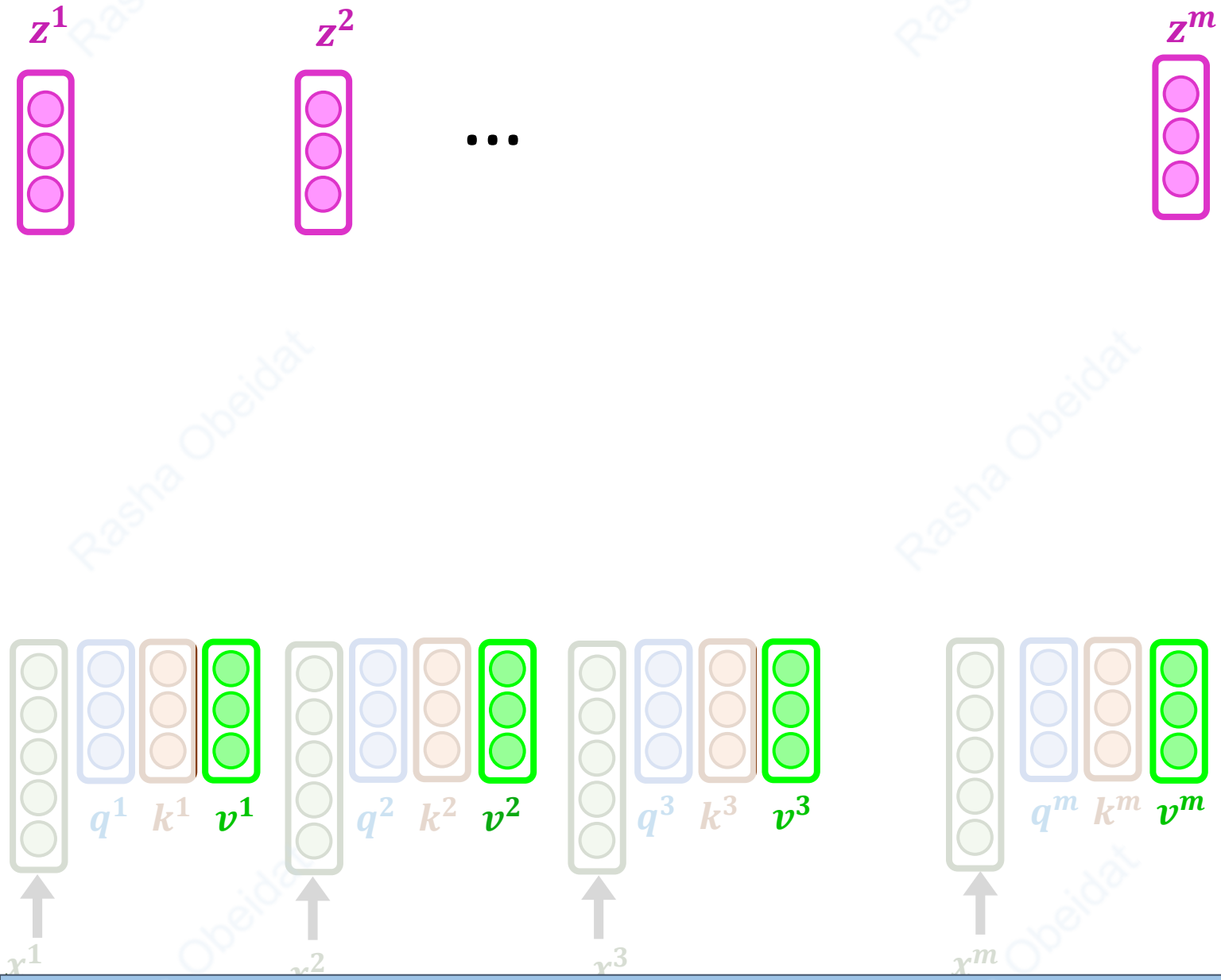
Compute Weighted Sum

$$z^m = \alpha^{m,1} \cdot v^1 + \alpha^{m,2} \cdot v^2 + \alpha^{m,3} \cdot v^3 + \cdots + \alpha^{m,m} \cdot v^m$$



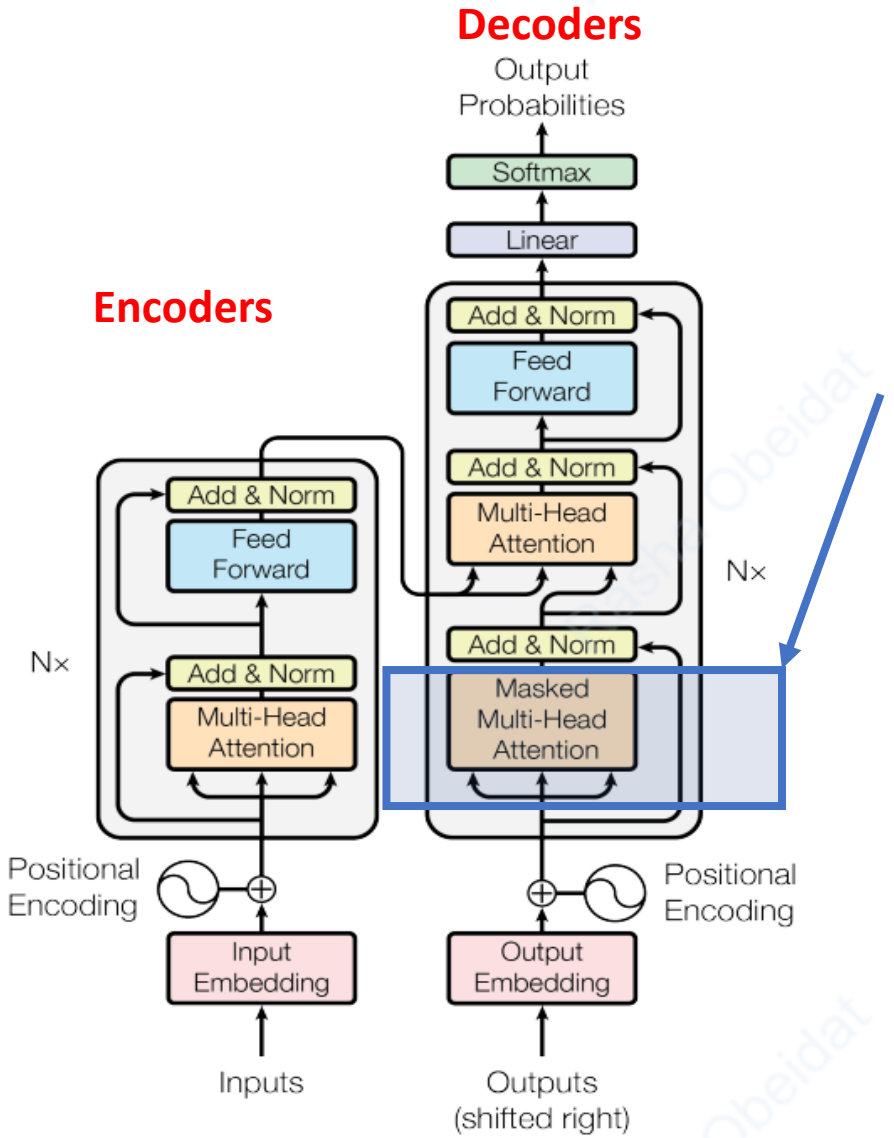


Masked Self-Attention



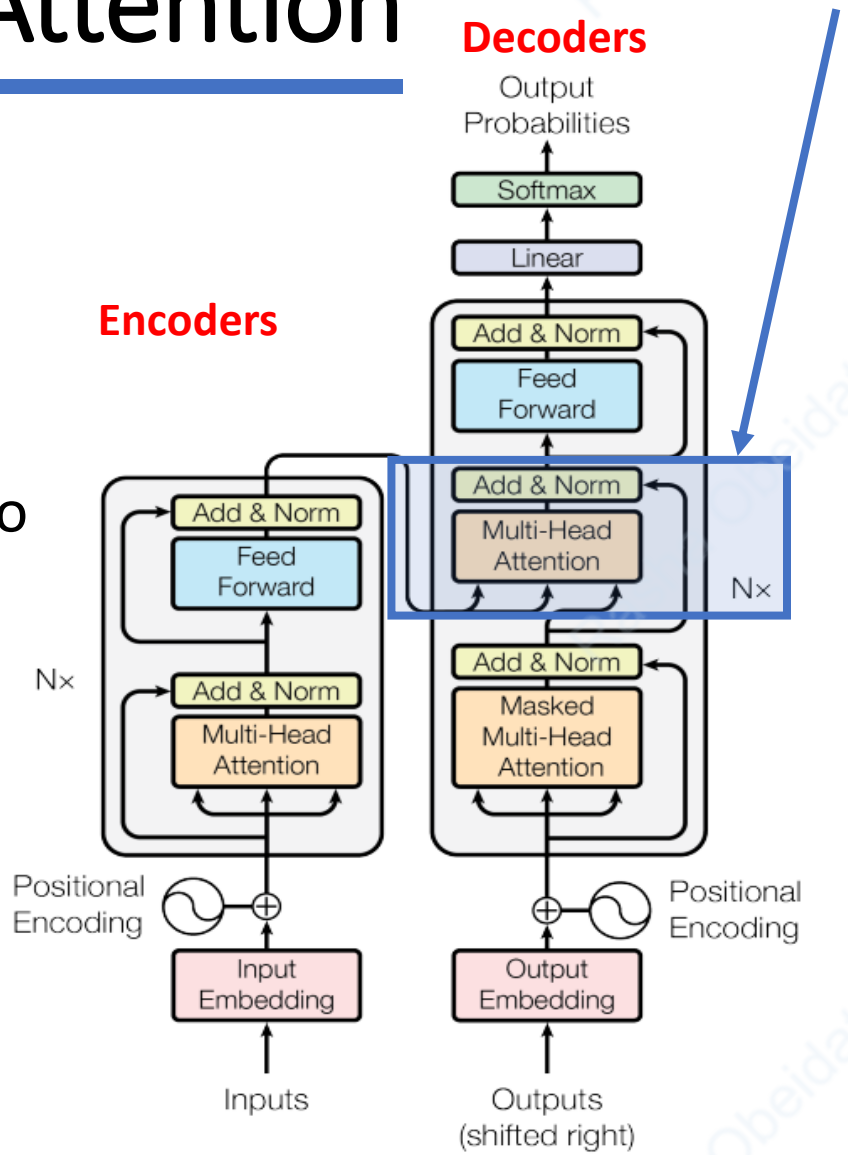
Decoder's Masked Multi-Head Self-Attention

- To summarize:
- Masked multi-head attention ensures that the decoder operates in an **auto-regressive manner**, just like during inference without accessing "future" information.
- **An auto-regressive decoder** refers to a model that generates output ***sequentially***, predicting one token at a time based on the tokens generated so far
- For example . For the sequence " يحب ابني الاصغر " when generating the word " ابني ", The decoder can only attend to "ابني" and " يحب " but not " الاصغر" or any future tokens.



Encoder-Decoder Multi-head Attention

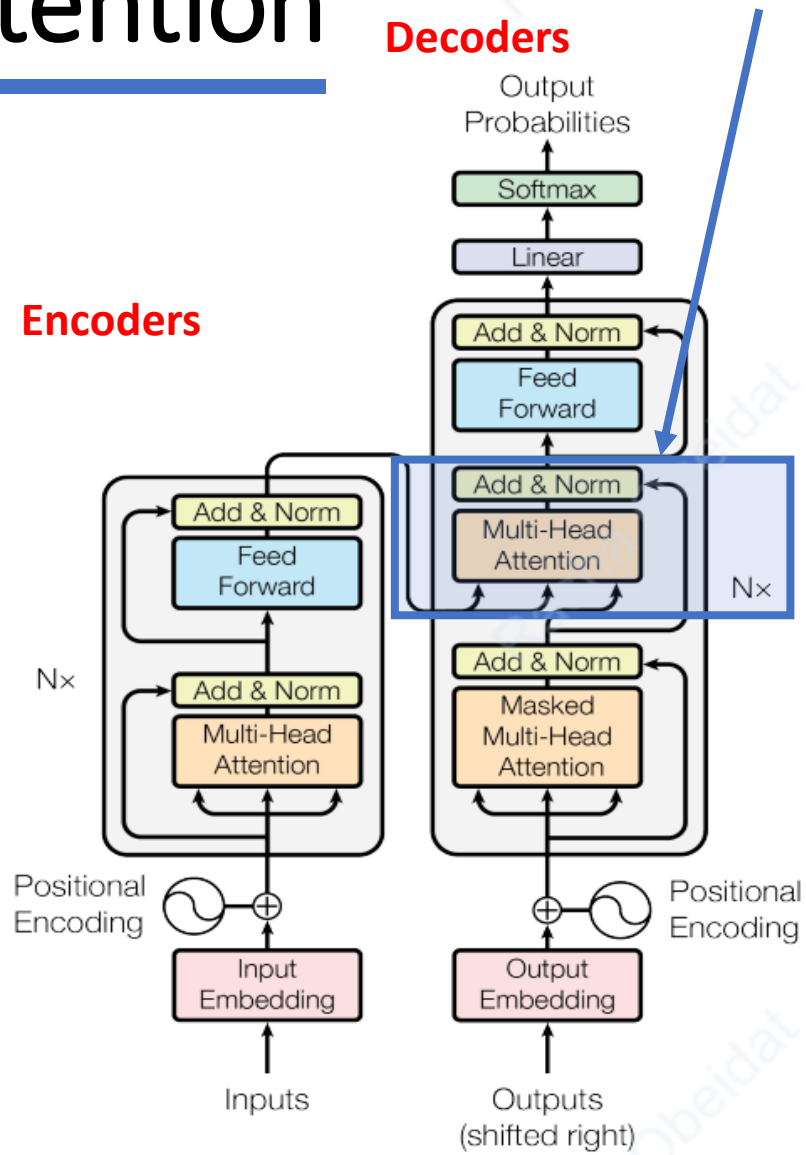
- At each decoding step, the decoder attends to the **entire encoder output**, which contains contextual representations of **all words in the input sentence**.
- This allows the decoder to focus on **relevant parts of the input**, similar to the attention mechanism added to seq2seq.
- In contrast, self-attention in the encoder compares **each word with all other words in the same input** to model internal dependencies.



Encoder-Decoder Multi-head Attention

Details:

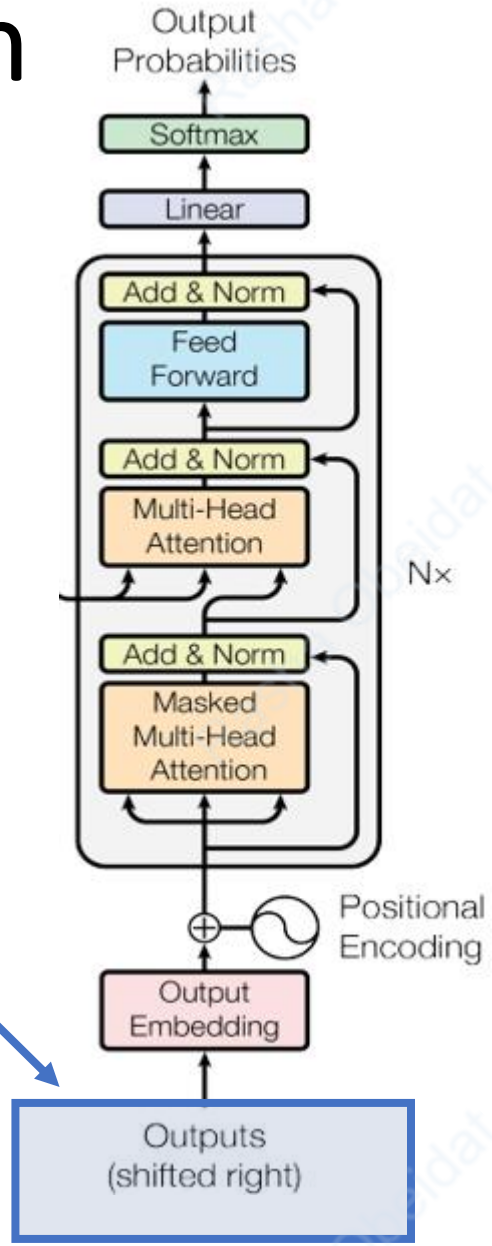
- In the **decoder**, at each decoding step, there is a special **encoder-decoder attention layer**. In this layer:
  - **Queries (Q)** are generated from the decoder's own hidden states.
  - **Keys (K)** and **Values (V)** come from the encoder's output representations of the input sentence.
  - The decoder **computes attention scores** between its Queries and the Keys from the encoder to decide **which parts of the input to focus on** when generating the next word.
- **For example:**
  - To generate the **first word**, the decoder computes attention scores by comparing the query (based on the special start symbol <s>) with each word representation (key) from the encoder.
  - These attention scores are then used to weight the **Values (V)** from the encoder outputs, producing a decoder-aware context vector for predicting the next token — similar to seq2seq models with attention.



Encoder-Decoder Multi-head Attention

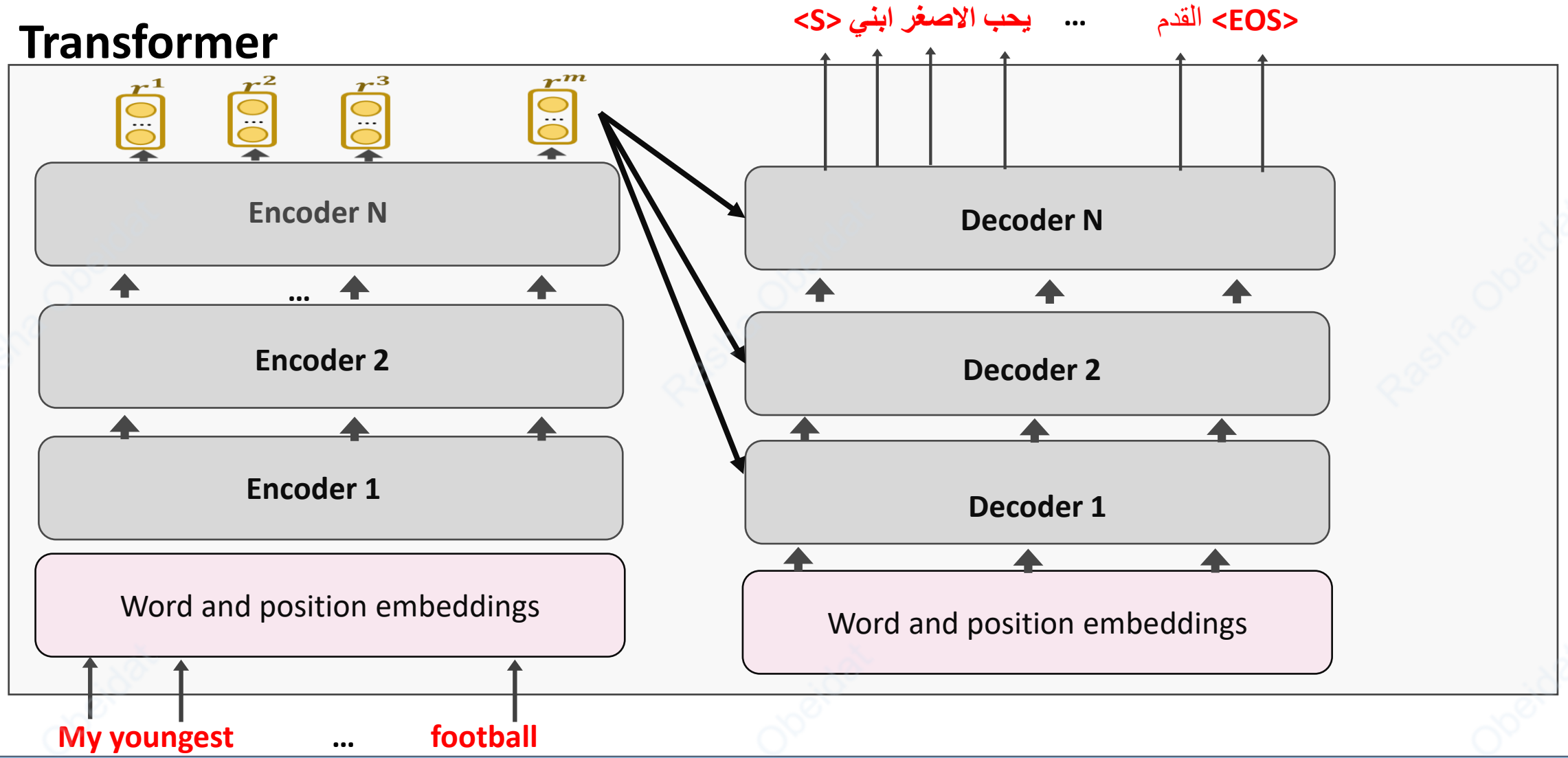
- What is the meaning of **“output shifted right”** in transformer’s decoder?

  It means that the target sequence shifted one position to the right, starting with the special start-of-sequence token (often <S>)



Transformer for MT

Loss Function: cross-entropy (predicting translated word)



Transformer for MT

Training Objective:

- The Transformer is trained to minimize **cross-entropy loss**, which measures the difference between the predicted and true token probabilities.

Teacher Forcing:

- During training, the true target tokens are provided to the decoder to condition the next token prediction, ensuring stable and efficient learning.

Inference with Beam Search:

- During inference, **beam search** is often used to explore multiple possible translations and select the most likely one based on the model's predictions.



Rasha Obeidat